A template/ approach to Reproducible Generative Research

Daniel Ari Friedman
Active Inference Institute
daniel@activeinference.institute
ORCID: 0000-0001-6232-9096
DOI: 10.5281/zenodo.20419007

2026-06-27

BEGINNING OF TRANSMISSION

State: unpublished / pending pairing
Pairing: pending — unresolved: - GitHub release URL: `pending` - PDF SHA-256: `pending`

Release metadata

Field	Value
Title	A template/ approach to Reproducible Generative Research
Version	2.0.0
Concept DOI	10.5281/zenodo.20419007
GitHub	docxology/template_template
Zenodo	https://zenodo.org/records/20419007
SHA-256	pending
SHA-512	pending

How to verify

- Scan **Integrity** QR and compare the embedded SHA-256 prefix to the table above.
- Scan **Zenodo** / **GitHub** QR codes and confirm they resolve to this release pairing.
- Full hashes and structured fields: `../data/transmission_manifest.json`.

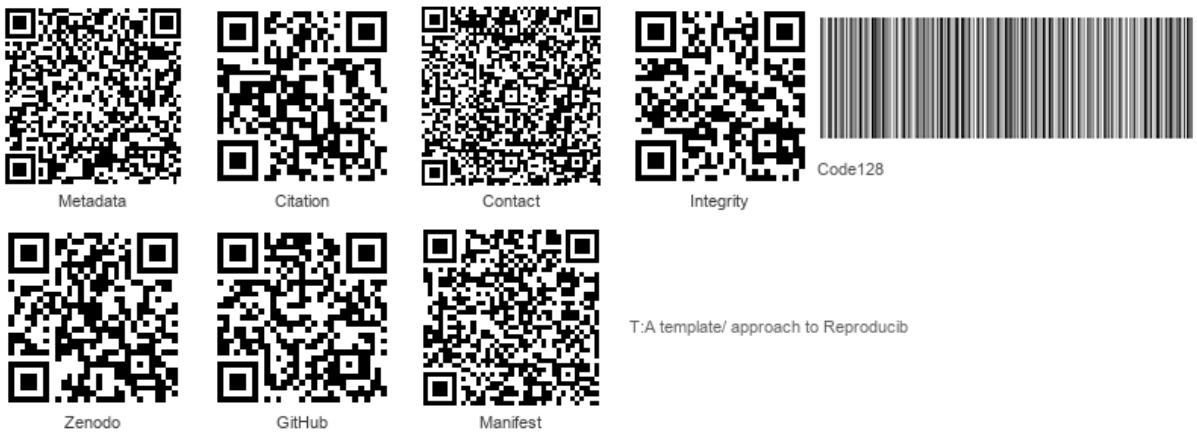


Figure 1: Integrity QR strip

Structured manifest: `../data/transmission_manifest.json`
Stego: off | overlays text | barcodes on | XMP on | manifest on → `./secure_run.sh`

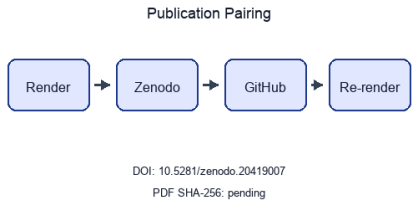


Figure 2: Publication pairing flow

Contents

1 Abstract **5**

2 Introduction **6**
2.1 Research Tools as Epistemic Infrastructure . . . 6
2.2 Related Work . . . 6
2.2.1 Workflow Managers . . . 6
2.2.2 Literate Programming and Publication Tools . . . 6
2.2.3 Containerization and Environment Capture . . . 7
2.2.4 Best-Practice Frameworks and Data Standards . . . 7
2.2.5 The Gap . . . 8
2.2.6 Summary: Gaps Left by Existing Tools . . . 8
2.3 `template/`: An Integrated Solution . . . 8
2.4 Scope and Contributions . . . 9
2.5 Paper Organization . . . 10

3 Methods **11**
3.1 The Two-Layer Architecture . . . 11
3.2 The Standalone Project Paradigm . . . 11
3.3 The Thin Orchestrator Pattern . . . 12
3.4 DAG Pipeline Declared by `pipeline.yaml` . . . 13
3.4.1 Stage Highlights . . . 13
3.4.2 Interactive Orchestration . . . 14
3.5 Documentation Duality and AI Collaboration . . . 15
3.6 Agentic Skill Architecture . . . 15
3.6.1 The Three-Tier Skill Protocol . . . 15
3.6.2 Module Skill Coverage . . . 16
3.6.3 MCP Server Mapping . . . 16
3.7 FAIR Alignment and Research Infrastructure as Code . . . 17
3.7.1 Principle-by-Principle Alignment . . . 17
3.7.2 Infrastructure as Code for Research . . . 17
3.8 Quality Assurance . . . 18
3.8.1 Zero-Mock Testing Policy . . . 18
3.8.2 Coverage Thresholds . . . 18
3.8.3 Test Suite Composition . . . 18
3.8.4 Visualization Standards . . . 18

4 Results **20**
4.1 Multi-Project Pipeline Execution . . . 20
4.2 Infrastructure Test Suite . . . 20
4.3 Infrastructure Module Inventory . . . 20
4.4 Agentic Skill Documentation Coverage . . . 22

4.5 DAG Reference (Declarative Executor) . . . 22
4.6 Steganographic Performance Boundary . . . 23
4.7 Self-Referential Analysis . . . 23
4.8 Comparative Feature Analysis . . . 23
4.9 Test Quality Metrics . . . 28

5 Discussion **29**
5.1 The Zero-Mock Tradeoff . . . 29
5.1.1 When Mocks Are Not the Problem . . . 29
5.1.2 Practical Implementation . . . 29
5.2 Scalability: From 1 to N Projects . . . 30
5.2.1 Multi-Project Orchestration . . . 30
5.2.2 Scaling Metrics . . . 30
5.3 Comparison to Existing Tools . . . 32
5.3.1 FAIR4RS Evolution (2024–2026) . . . 32
5.4 The AI Collaboration Model . . . 33
5.5 The Learning Curve . . . 33
5.6 Limitations . . . 33
5.7 Future Directions . . . 35
5.8 Conclusion . . . 35

6 Infrastructure Module Reference **37**
6.1 Alphabetical summaries . . . 38
6.1.1 `infrastructure.autoresearch` (11 files) . . . 38
6.1.2 `infrastructure.benchmark` (4 files) . . . 38
6.1.3 `infrastructure/config` (non-package subdirectory) . . . 39
6.1.4 `infrastructure.core` (201 files) . . . 39
6.1.5 `infrastructure.doctor` (14 files) . . . 39
6.1.6 `infrastructure.docker` (0 files) . . . 39
6.1.7 `infrastructure.documentation` (23 files) . . . 39
6.1.8 `infrastructure.formal_contracts` (5 files) . . . 39
6.1.9 `infrastructure.fonds` (6 files) . . . 39
6.1.10 `infrastructure.llm` (57 files) . . . 39
6.1.11 `infrastructure.metadata` (3 files) . . . 39
6.1.12 `infrastructure.methods` (10 files) . . . 39
6.1.13 `infrastructure.orchestration` (14 files) . . . 39
6.1.14 `infrastructure.project` (116 files) . . . 40
6.1.15 `infrastructure.prose` (9 files) . . . 40
6.1.16 `infrastructure.provenance` (9 files) . . . 40
6.1.17 `infrastructure.publishing` (85 files) . . . 40
6.1.18 `infrastructure.reference` (16 files) . . . 40
6.1.19 `infrastructure.rendering` (120 files) . . . 40
6.1.20 `infrastructure.reporting` (411 files) . . . 40
6.1.21 `infrastructure.research` (3 files) . . . 40
6.1.22 `infrastructure.rules` (6 files) . . . 40
6.1.23 `infrastructure.scientific` (4 files) . . . 40
6.1.24 `infrastructure.search` (77 files) . . . 40
6.1.25 `infrastructure.sia` (10 files) . . . 41
6.1.26 `infrastructure.skills` (8 files) . . . 41
6.1.27 `infrastructure.steganography` (13 files) . . . 41
6.1.28 `infrastructure.tools` (6 files) . . . 41
6.1.29 `infrastructure.transmission` (6 files) . . . 41
6.1.30 `infrastructure.validation` (87 files) . . . 41
6.1.31 `infrastructure/logrotate.d` (0 files) . . . 41

7 Security and Provenance **42**
7.1 Threat Model . . . 42
7.2 Steganographic Layers . . . 42
7.2.1 Layer 1: PDF Metadata Injection . . . 42
7.2.2 Layer 2: Cryptographic Hashing . . . 42
7.2.3 Layer 3: Alpha-Channel Overlay, Footer, and Invisible Text . . . 42
7.2.4 Layer 4: QR and Barcode Injection . . . 43
7.3 The `secure_run.sh` Orchestrator . . . 43
7.4 Tamper Detection . . . 43
7.5 Limitations . . . 43
7.6 Relationship to Software Supply Chain Integrity . . . 44
7.7 Relationship to FAIR and Formal Provenance Standards . . . 44

8 Appendices **45**
8.1 Appendix: Pipeline Stage Reference . . . 45
8.1.1 Preparation and analysis . . . 45
8.1.2 Search, provenance and rendering . . . 45
8.1.3 Review and delivery . . . 46
8.1.4 Optional packaging and publication . . . 46
8.2 Appendix: Configuration Reference . . . 48
8.3 Appendix: Repository Directory Structure . . . 50
8.4 Appendix: Exemplar Project Summary . . . 51
8.5 Appendix: Documentation Inventory . . . 52
8.6 Appendix: Comparative Tool Matrix . . . 53

1 Abstract

The reproducibility problem in computational research is partly structural: research artifacts are often distributed across editors, notebooks, scripts, and manual publication steps without a shared mechanism that keeps code, data, and manuscript synchronized. Existing tools address important parts of this lifecycle: workflow managers orchestrate computation, literate-programming systems render documents, and data-versioning tools track artifacts. `template/` combines those concerns with test and provenance gates by applying Infrastructure as Code to the research lifecycle. Its Two-Layer Architecture separates 31 infrastructure subdirectories (28 importable Python packages, about 1334 modules, validated by about 12,548 tests) from self-contained project workspaces. A YAML-declared pipeline (17 stages; 10 in a default full run) coordinates environment checks, test execution, analysis, Pandoc/XeLaTeX rendering, SHA-256 hashing with optional steganographic watermarking, structural validation, and optional LLM review. The declared quality floors—90% for project source and 60% for infrastructure source—are injected from executable configuration rather than copied into the manuscript. Documentation Duality equips repository directories with human-facing `README.md` and agent-facing `AGENTS.md` files; skill-enabled infrastructure packages additionally expose structured `SKILL.md` descriptors so agents can discover supported capabilities from versioned interfaces.
The generated public exemplar roster (`templates/template_active_inference`, `templates/template_advanced_literature_review`, `templates/template_autopoiesis`, `templates/template_autoresearch_project`, `templates/template_autoscientists`, `templates/template_code_project`, `templates/template_data_descriptor`, `templates/template_eda_notebook`, `templates/template_experiment_tree`, `templates/template_formal`, `templates/template_gold_refinement`, `templates/template_literature_meta_analysis`, `templates/template_madlib`, `templates/template_methods_paper`, `templates/template_newspaper`, `templates/template_pitch_deck`, `templates/template_pools_rules_tools`, `templates/template_prose_project`, `templates/template_redacted_report`, `templates/template_registered_report`, `templates/template_search_project`, `templates/template_sia`, `templates/template_storybook`, `templates/template_template`, `templates/template_textbook`) supplies heterogeneous control-positive layouts, including optimization (`template_code_project`, 263 discovered tests), prose (`template_prose_project`, 165 discovered tests), and AutoResearch readiness (`template_autoresearch_project`, 318 discovered tests). Each declares the same 90% project-source floor and consumes the shared pipeline without project-to-project imports. This manuscript adds a reflexive artifact: `template_template` (158 discovered tests) follows the same analysis and render path while injecting repository counters and figures from live introspection. The result is a self-documenting build whose quantitative repository claims are generated artifacts, whose manuscript claims are checked against an evidence registry, and whose output can be validated and optionally watermarked. A versioned comparative matrix situates `template/` against peer tools across the capabilities reported in the Results section. Code is released under the Apache License 2.0 at `github.com/docxology/template`; the work remains open-ended.

2 Introduction

2.1 Research Tools as Epistemic Infrastructure

Scientific research operates through a layered ecology of tools, documents, and practices—each shaping what can be known, communicated, and verified. When these layers are fragmented, manuscripts, data, code, and figures can drift out of alignment, creating gaps that are structural rather than incidental. Survey and economic evidence documents the practical cost of failed reproduction attempts [Baker, 2016, Freedman et al., 2015], but reproducibility is only the most visible face of a broader problem. Research software engineering, epistemic integrity, and the coordination between human researchers and AI collaborators all depend on whether the tools that produce research artifacts can themselves be transparent, testable, and self-documenting. Nosek et al. [Nosek et al., 2018] argue that committing to analytical plans before data collection is a necessary structural reform; we extend this logic to the research build pipeline.

One root cause is fragmentation—of attention (in the mind) as well as in the cyberphysical niche (documents, versions, and reproducibility artefacts). A typical research project scatters its artifacts across disconnected tools: LaTeX editors [Lamport, 1994] for prose, Jupyter notebooks for analysis, ad-hoc shell scripts for figure generation, and manual copy-paste for integrating results into manuscripts. Each boundary between tools is a potential locus of desynchronization. The version of a figure embedded in a PDF may not match the code that ostensibly generated it, while tests may exercise analysis code without exercising the rendering path. Pimentel et al. [Pimentel et al., 2019] studied a corpus described as 1.4 million Jupyter notebooks; in the valid execution subset, 24.11% ran without errors and 4.03% reproduced the recorded results. Those distinctions matter: execution success is not equivalent to result reproduction. Peng [Peng, 2011] argues that reproducibility in computational science requires, at minimum, that the data and code underlying a published result be available for independent verification. Barba [Barba, 2018] further documents how "reproducibility," "replicability," and "repeatability" carry conflicting definitions across disciplines, complicating cross-field standards.

2.2 Related Work

Gentleman and Temple Lang [Gentleman and Temple Lang, 2007] introduced the concept of a *research compendium*—a single unit of scholarly communication bundling code, data, and narrative. This vision has driven two decades of tooling, which can be grouped into four categories: workflow managers, literate programming systems, containerization approaches, and best-practice frameworks.

2.2.1 Workflow Managers

Snakemake [Köster and Rahmann, 2012] uses a rule-based, Python-derived DSL to specify computational workflows as directed acyclic graphs of file-producing steps. It supports containerized execution and an extension architecture for execution and storage backends, while its central scope remains computational pipeline orchestration rather than this repository's combined manuscript, test, and document-provenance contract.

Nextflow [Di Tommaso et al., 2017] employs a dataflow programming paradigm with native support for container-based execution across heterogeneous computing environments (local, SLURM, AWS). Like Snakemake, Nextflow excels at bioinformatics pipeline parallelism but does not address manuscript production, document integrity, or the testing–publication coupling that characterizes research reproducibility.

CWL (Common Workflow Language) [Amstutz et al., 2016] provides a portable, YAML-based standard for describing computational workflows and their dependencies. Its strength lies in interoperability across execution engines (cwltool, Toil, Arvados), but it requires external tooling for manuscript generation and offers no built-in testing or provenance framework.

2.2.2 Literate Programming and Publication Tools

Knuth's literate programming [Knuth, 1984] established the principle that programs should be authored as documents intended for human comprehension. Schulte et al. [Schulte et al., 2012] extended this to

multi-language computing environments (Org-mode), demonstrating that literate programming could span languages and output formats.

Quarto [Allaire et al., 2024] extends the R Markdown tradition to support Python, Julia, and Observable, rendering to PDF, HTML, and Word. Quarto integrates code execution with document rendering, achieving a modern form of literate programming, but it does not enforce testing thresholds, manage multi-project repositories, or provide cryptographic provenance.

Jupyter Book [Kluyver et al., 2016] builds on Jupyter notebooks to produce publication-quality documents via Sphinx. While powerful for interactive exploration, Jupyter's notebook format introduces execution-order fragility [Pimentel et al., 2019] and does not naturally support the separation of logic from orchestration that characterizes maintainable research software.

R Markdown [Xie et al., 2018] pioneered knitr-based dynamic documents that weave code and prose. Its ecosystem is rich but R-centric, and it lacks the multi-project management, infrastructure testing, and provenance embedding that characterize `template/`.

Typst [Mädje and Haug, 2023] is an emerging markup-based typesetting system with incremental compilation and a programmable scripting layer. While Typst offers faster compilation than LaTeX and a more modern authoring experience, it does not integrate testing, provenance, or multi-project pipeline management.

2.2.3 Containerization and Environment Capture

Docker [Boettiger, 2015] addresses reproducibility at the environment level—packaging operating system, libraries, and code into portable containers. While Docker solves the "works on my machine" problem, containerization is complementary to, not a replacement for, the architectural concerns addressed here: Docker does not enforce testing, embed provenance, or manage multi-project manuscript workflows.

2.2.4 Best-Practice Frameworks and Data Standards

Wilson et al. [Wilson et al., 2017] define "good enough" practices for scientific computing, emphasizing version control, testing, and documentation. Sandve et al. [Sandve et al., 2013] propose ten rules for reproducible computational research. Piccolo and Frampton [Piccolo and Frampton, 2016] systematically survey tools for computational reproducibility, finding that environment isolation, workflow automation, and documentation generation address complementary but non-overlapping reproducibility concerns—yet no single tool unifies all three. Stodden et al. [Stodden et al., 2016] advocate for enhanced computational method transparency. The FAIR principles [Wilkinson et al., 2016]—Findable, Accessible, Interoperable, Reusable—establish a standard for data stewardship that has been widely adopted by funding agencies and journals. Lamprecht et al. [Lamprecht et al., 2020] formalize "Towards FAIR Principles for Research Software," providing the conceptual scaffolding that Barker et al. [Barker et al., 2022] would soon operationalize as the FAIR4RS initiative—recognizing that software has execution, composability, and dependency-management requirements that data-centric FAIR does not address. Cohen et al. [Cohen et al., 2021] characterize the four pillars of research software engineering (software sustainability, software quality, community building, and policy advocacy), situating formal testing and provenance practices within a broader RSE governance framework. Garijo et al. [Garijo et al., 2024] operationalize FAIR4RS through the FAIRsoft evaluator, an automated assessment framework that scores research software against 17+ quality indicators including executability, metadata richness, and documentation completeness. Goble et al. [Goble et al., 2020] extend FAIR to computational workflows specifically, arguing that workflow provenance requires first-class treatment in scientific computing infrastructure. Nüst et al. [Nüst et al., 2017] introduce the *executable research compendium* (ERC), extending Gentleman and Temple Lang's compendium concept with containerized, interactive reproduction environments. The W3C PROV data model [Moreau and Missier, 2013] provides a formal vocabulary for expressing provenance records, while in-toto [Torres-Arias et al., 2019] provides a framework for end-to-end software supply chain integrity verification, and SLSA [Open Source Security Foundation, 2023] (Supply-chain Levels for Software Artifacts) extends this to graduated, attestation-based supply-chain security levels for build pipelines. These frameworks articulate *what* reproducible research requires but do not provide an integrated *how*—they lack the tooling, enforcement mechanisms, and architectural patterns that translate standards into practice.

2.2.5 The Gap

Despite advances in FAIR4RS principles [Barker et al., 2022, Lamprecht et al., 2020], automated FAIR software assessment [Garijo et al., 2024], FAIR computational workflow standards [Goble et al., 2020], supply-chain attestation frameworks [Torres-Arias et al., 2019, Open Source Security Foundation, 2023], and the preregistration revolution [Nosek et al., 2018], no existing system integrates six cross-cutting concerns into a single enforced pipeline: (1) end-to-end pipeline orchestration with testing enforcement, (2) multi-format manuscript rendering, (3) cryptographic provenance embedding, (4) multi-project repository management, (5) FAIR-aligned software stewardship, and (6) AI-agent collaboration via structured documentation. Each existing framework addresses a subset; none provides the unified enforcement mechanism. The detailed tool-by-tool comparison is developed in the Comparison to Existing Tools section; the summary table below captures the gap landscape.

2.2.6 Summary: Gaps Left by Existing Tools

The six concerns identified above map onto existing tool categories as follows:

Gap	Partially Addressed By	Not Addressed By
Pipeline orchestration	Snakemake, Nextflow, CWL	Quarto, Jupyter Book, R Markdown, Typst, Overleaf, Prism
Manuscript rendering	Quarto, Jupyter Book, R Markdown, Typst, Overleaf, Prism	Snakemake, Nextflow, CWL, DVC
Testing enforcement	—	All existing tools
Cryptographic provenance	SLSA (build-level attestation only)	All research-focused tools
Multi-project management	—	All existing tools
AI-agent documentation	Overleaf (partial co-author AI), Prism (partial context reasoning)	All pipeline/workflow tools
Agentic skill protocol (MCP-aligned)	—	All existing tools

No existing system addresses all six concerns within a single enforced pipeline.

2.3 `template/`: An Integrated Solution

`template/` was conceived as a structural antidote to this fragmentation. Rather than adding reproducibility as an afterthought—a Docker container wrapping an already-disjointed workflow [Boettiger, 2015]—the template enforces integrity at the architectural level. It realizes Gentleman and Temple Lang's research compendium vision [Gentleman and Temple Lang, 2007] at repository scale, bundling code, data, tests, manuscripts, and provenance into a single, pipeline-enforced system with version-controlled infrastructure [Ram, 2013]. It stands on four primary pillars:

1. **Ergonomic Modularity**: A Two-Layer Architecture cleanly separates globally shared infrastructure (logging, rendering, validation, steganography) from project-specific logic (manuscripts, scripts, data). 31 infrastructure subdirectories (28 importable packages) comprising about 1334 Python modules provide reusable services; projects consume them without modification.
2. **Execution Integrity**: Pipeline advancement is contingent on test passage. The configured source-coverage floors are 60% for infrastructure and 90% for each project. The static gate rejects prohibited mock-framework imports, while the suites exercise real filesystem and subprocess behavior and use local protocol servers or explicitly enabled live services for network boundaries. about 12,548 infrastructure tests and 6339+ project tests are discovered by the introspection build.
3. **Automated Provenance**: Steganographic watermarking and cryptographic hashing are integrated directly into the rendering pipeline. Every generated PDF carries SHA-256 (and, when enabled, SHA-512) fingerprints, an alpha-channel watermark (configurable text or QR overlay), per-page provenance

footers and barcode strips, and embedded PDF metadata recording the build timestamp, document identifier, and Git commit hash. Provenance is not asserted by policy; it is enforced by architecture.

4. **AI-Agent Collaboration and Skill-Based Agentic Operations**: A three-tier documentation architecture enables AI agents to operate at every level of the system. At the system level, root `AGENTS.md` and `.agents/` define shared constraints and generic workflows; client entrypoints reference these shared contracts. At the structural level, `AGENTS.md` files in documented directories expose local API surfaces, file inventories, and integration contracts. At the module level, `SKILL.md` files—written to a discoverable YAML+Markdown schema aligned with the Model Context Protocol [Anthropic, 2024]—describe skill-enabled modules and exemplars as reusable capabilities.

An agent invoking `infrastructure.rendering` does not need to read source code: it reads the `rendering/SKILL.md`, which declares the module's name, description, activation metadata in YAML frontmatter and imports and example invocations in its Markdown body. This architecture is the practical realization of the skill-library paradigm established in the agent literature: Yao et al.'s ReAct framework [Yao et al., 2023] demonstrated that interleaving reasoning traces with tool invocations dramatically improves LLM reliability; Schick et al.'s Toolformer [Schick et al., 2023] showed that self-supervised tool use can be bootstrapped from natural language; Wang et al.'s Voyager [Wang et al., 2023] proved that growing skill libraries enable open-ended autonomous exploration in complex environments. `template/` instantiates this vision in the domain of scientific research infrastructure: each `SKILL.md` is a Voyager-style skill, each pipeline stage is a ReAct action, and the full infrastructure layer constitutes a composable, protocol-aligned skill library for scientific computation.

2.4 Scope and Contributions

This paper is itself a product of the template it describes. The metrics populating its tables were computed by the introspection module documented in the Methods; the figures were rendered by the visualization code validated by the test suite described in Quality Assurance; the PDF carrying these words was assembled by the same YAML-declared pipeline whose architecture is the subject of the Results. This self-productive loop—where the system that is described is also the system that produces the description—is not incidental but structural, a concrete demonstration that `template/` can sustain the full lifecycle from source code to published artifact within a single, version-controlled, pipeline-enforced repository. Our contributions are:

- A formal description of the Two-Layer Architecture and Standalone Project Paradigm that enables N independent research projects to share infrastructure without coupling.
- A detailed specification of the 17-stage DAG in `infrastructure/core/pipeline/pipeline.yaml`: default full runs use 10 stages; `--mode core` runs 8; opt-in ebook, docxplus, metadata, bundle, and archival stages via `--tags`.
- A comparative analysis positioning `template/` against nine peer tools—Snakemake, Nextflow, CWL, Quarto, Jupyter Book, R Markdown, DVC, Overleaf, OpenAI Prism—across fourteen feature dimensions, demonstrating that `template/` uniquely bundles the eleven capabilities it supports (testing, provenance, documentation, and orchestration enforcement) of the fourteen surveyed dimensions enumerated in §Results.
- An empirical evaluation across the canonical exemplar projects under `projects/templates/` (`templates/template_active_inference`, `templates/template_advanced_literature_review`, `templates/template_autopoiesis`, `templates/template_autoresearch_project`, `templates/template_autoscientists`, `templates/template_code_project`, `templates/template_data_descriptor`, `templates/template_eda_notebook`, `templates/template_experiment_tree`, `templates/template_formal`, `templates/template_gold_refinement`, `templates/template_literature_meta_analysis`, `templates/template_madlib`, `templates/template_methods_paper`, `templates/template_newspaper`, `templates/template_pitch_deck`, `templates/template_pools_rules_tools`, `templates/template_prose_project`, `templates/template_redacted_report`, `templates/template_registered_report`, `templates/template_search_project`, `templates/template_sia`, `templates/template_storybook`, `templates/template_template`, `templates/template_textbook`)—including this meta manuscript from `projects/templates/template_template`, which exercises introspection-derived metrics.
- A security analysis of the steganographic provenance layer, including a formal threat model and tamper-

detection capabilities aligned with the W3C PROV data model [Moreau and Missier, 2013] and SLSA [Open Source Security Foundation, 2023].
- An open-source reference implementation available at `github.com/docxology/template`.

2.5 Paper Organization

The Methods describe the Two-Layer Architecture, Thin Orchestrator pattern, pipeline stages, and AI collaboration model. Results present quantitative metrics from multi-project execution, coverage analysis, and steganographic benchmarks. The Discussion addresses the Zero-Mock tradeoff, scalability implications, a detailed tool comparison, and future directions. The Infrastructure Module Reference provides detailed documentation for all 31 subdirectories. Security and Provenance describes the steganographic and cryptographic integrity layer. The Appendices provide pipeline, configuration, and comparative references.

3 Methods

The template/ architecture is deliberately bifurcated into a globally shared infrastructure/ layer and project-specific projects/ silos. This section describes the four core design patterns, the YAML-declared pipeline (17 stages; default full 10) pipeline that operationalizes them, and the AI collaboration model that distinguishes this system from conventional research templates.

3.1 The Two-Layer Architecture

The repository is organized into two strictly separated layers:
Infrastructure Layer (infrastructure/): 31 infrastructure subdirectories—28 of them independently-importable Python packages—comprising about 1334 modules and providing reusable services. Each importable package has its own __init__.py, AGENTS.md, and README.md, and exports a well-defined public API (the remaining subdirectories, e.g. config/, hold shared configuration). The infrastructure layer knows nothing about any specific project—it provides generic capabilities (logging, rendering, validation, steganography) that any project may consume.
Project Layer (projects/): Self-contained research workspaces. Each project directory contains:

Directory	Purpose
manuscript/	Markdown chapters and config.yaml
scripts/	Thin orchestrator scripts (Stage 02)
src/	Project-specific Python modules
tests/	Project-specific test suite
data/	Input datasets and generated data
output/	Pipeline artifacts: PDF, figures, reports, logs
docs/	Project-specific architecture documentation

The two layers communicate exclusively through Python imports and filesystem paths. No project modifies infrastructure code; no infrastructure module references a specific project by name (except via runtime project discovery).

3.2 The Standalone Project Paradigm

Projects are designed to be completely self-contained. Adding a new project requires no changes to the infrastructure layer, no modifications to pyproject.toml, and no updates to the pipeline orchestrator. A project is automatically discovered if and only if it satisfies two conditions:

1. It exists as a subdirectory of projects/.
2. It contains the file manuscript/config.yaml.

This paradigm enables horizontal scaling: N researchers can maintain N independent projects within a single repository, sharing infrastructure without coupling. Each project declares its own testing tolerances, manuscript metadata, LLM review preferences, and rendering configuration in its config.yaml. The system currently hosts its public canonical exemplars under projects/templates/ (templates/template_active_inference, templates/template_advanced_literature_review, templates/template_autopoiesis, templates/template_autoresearch_project, templates/template_autoscientists, templates/template_code_project, templates/template_data_descriptor, templates/template_eda_notebook, templates/template_experiment_tree, templates/template_formal, templates/template_gold_refinement, templates/template_literature_meta_analysis, templates/template_madlib, templates/template_methods_paper, templates/template_newspaper, templates/template_pitch_deck, templates/template_pools_rules_tools, templates/template_prose_project, templates/template_redacted_report, templates/template_registered_report, templates/template_search_project, templates/template_sia, templates/template_storybook, templates/template_template, templates/template_textbook), including this meta-manuscript at projects/templates/template_template/.

3.3 The Thin Orchestrator Pattern

All scripts in `scripts/` (both infrastructure-level and project-level) follow the Thin Orchestrator pattern [Gamma et al., 1995]:

- **No domain logic**: Scripts contain zero algorithmic implementation. They import functions from `src/` modules and wire them to infrastructure services.
- **Configuration-driven**: Behavior is parameterized by `config.yaml`, not by hardcoded values.
- **Stateless**: Scripts read inputs, call functions, write outputs. They maintain no persistent state between invocations.
- **Logged**: Every significant action is logged via `infrastructure.core.logging.utils.get_logger`.

This pattern ensures that all testable logic lives in `src/` where it is subject to the Zero-Mock testing policy, while scripts remain thin enough to be audited by visual inspection. The separation draws on the Mediator pattern from Gamma et al. [Gamma et al., 1995], where scripts mediate between infrastructure services and project-specific code without implementing any logic of their own.

To make this concrete, the following contrasts the anti-pattern with the correct pattern:

```
# ANTI-PATTERN: domain logic embedded in script
def calculate_average(data):          # ← never put computation here
    return sum(data) / len(data)

result = calculate_average([1, 2, 3])

# CORRECT PATTERN: script imports from src/ and only wires
from projects.my_project.src.statistics import calculate_average

result = calculate_average([1, 2, 3])  # ← scripts wire, never compute
```

The critical property is that `calculate_average` in the correct pattern lives in a testable `src/` module, is covered by the Zero-Mock test suite, and can be independently imported, tested, and reused—whereas the anti-pattern buries logic in a script that is invisible to coverage tools.

3.4 DAG Pipeline Declared by `pipeline.yaml`

Single-project pipelines read `infrastructure/core/pipeline/pipeline.yaml`. `scripts/runner/execute_pipeline.py` expands the declarative DAG, applies tag filters (`--mode core` skips `llm` stages), checkpoints between nodes, then dispatches the declared `scripts/pipeline/stage_NN_*.py` scripts or builtin methods (`_run_clean_outputs`).

The YAML declares **17 stages**. Tag filtering selects **10 stages for a default full run** and **8 for core mode**. The overview below covers the default full selection: core stages plus live LLM stages. Telemetry settings are configuration metadata, not additional DAG nodes.

1. **Clean Output Directories** — wipes prior `projects/<name>/output/` + delivered `output/<name>/` paths so stale PDFs cannot satisfy validation.
2. **Environment Setup** (`scripts/pipeline/stage_00_setup.py`) — Python/uv probing, toolchain discovery, scaffolding directories, `PYTHONPATH` wiring.
3. **Infrastructure Tests** (`scripts/pipeline/stage_01_test.py`, `--infra-only`, `--infra-scope pipeline-smoke`) — focused real infrastructure contracts for the project pipeline. The full infrastructure suite and its configured >=60% coverage floor remain separate verification.
4. **Project Tests** (`scripts/pipeline/stage_01_test.py --project-only`) — isolated per-project suites with each project's declared floor (>=90% for this exemplar).
5. **Project Analysis** (`scripts/pipeline/stage_02_analysis.py`) — lexicographically ordered `projects/<name>/scripts/*.py`, each a thin orchestrator (`src/` does real work).
6. **PDF Rendering** (`scripts/pipeline/stage_03_render.py`) — Pandoc → XeLaTeX loop, bibliography assembly, injected variables from Stage 02 artefacts.
7. **Output Validation** (`scripts/pipeline/stage_04_validate.py`) — PDF structure, manifests, Markdown hygiene.
8. **LLM Manuscript Review** (`scripts/pipeline/stage_06_llm_review.py --reviews-only`; `tags: llm`) — executive + quality critiques via local Ollama; `allow_skip: true`.
9. **LLM Translations** (`scripts/pipeline/stage_06_llm_review.py --translations-only`; tags `llm`, same dependency edges) — multilingual abstract expansion.
10. **Copy Outputs** (`scripts/pipeline/stage_05_copy.py`) — reproducible snapshots into canonical `output/<project>/`.

The two LLM nodes share one script module with orthogonal CLI switches and depend on output validation. Full mode includes them; core mode excludes them. When a stage permits a skip, it must retain a fresh policy-bound reason receipt; unavailable required live acceptance remains blocked.

Executive reporting (`scripts/pipeline/stage_07_executive_report.py`) is a separate multi-project reporting operation, not an additional YAML node inside the single-project executor. `execute_multi_project.py` writes aggregate summary reports after project execution.

Topological order therefore differs slightly from declaration order (e.g., copy executes after validation even though stage `stage_05_copy.py` precedes `stage_06_llm_review.py` lexically).

3.4.1 Stage Highlights

Infrastructure and project tests. Separate invocations isolate the focused infrastructure smoke contract from each project's nonempty test suite and project-local coverage gate. Required test or coverage failures and unsuccessful runners block pipeline completion; configured failure tolerances cannot turn a nonzero child test result into success. The full infrastructure coverage floor (>=60%) is verified separately. An explicitly skippable stage may accept exit 2 only with a fresh reason receipt bound to the current run, stage, command, and plan. That recorded skip does not satisfy required live acceptance.

Stage 02 illustration. The analysis stage is deliberately concrete rather than a hypothetical diagram factory: each canonical project ships real behaviour at this node. `template_autoresearch_project` runs readiness validation; `template_search_project` merges remote literature JSON, generates scripted figures (`y_generate_search_figures.py`), and writes manifests; `template_code_project` emits optimization plots; and `template_prose_project` triggers structural validation scaffolding. The pipeline shape is identical across all four—only the Stage 02 payload differs—which is exactly what lets one orchestrator serve heterogeneous research domains. The live public roster is generated in `active_projects.md`.

3.4.2 Interactive Orchestration

3.4.2.1 `run.sh` Thin wrapper invoking `python -m infrastructure.orchestration`. Offers:

- per-project staged execution,
- chained digits (`234` shorthand),
- multi-project grid (`a`–`d` presets),
- graceful quit / resume parity with `scripts/README.md`.

Selecting **`d` alone** after a passing multi-project run exits immediately once summaries print—avoiding repetitive menu redraw.

3.4.2.2 `secure_run.sh` Executes Python `secure` path: standard pipeline artefact reproduction **then** invokes `run_secure_pipeline` for steganographic PDF hardening (`infrastructure.steganography`). Original PDFs stay immutable; hardened companions carry QR overlays plus hash manifests sidecars.

3.5 Documentation Duality and AI Collaboration

The Documentation Duality standard pairs two files in documented source directories; the live inventory reports actual coverage:

- **`README.md`**: Human-readable overview, quick-start instructions, and directory structure.
- **`AGENTS.md`**: Machine-readable technical specification optimized for AI coding assistants. Contains API tables, dependency graphs, implementation patterns, and architectural constraints.

This Documentation Duality standard serves two purposes. First, it ensures that both human researchers and AI agents can navigate the codebase efficiently—`AGENTS.md` files provide the structured context that language models need to make informed code modifications without hallucinating API signatures or violating architectural invariants. Second, it creates a self-documenting feedback loop: as AI agents modify the codebase, they update the corresponding `AGENTS.md` files, keeping documentation synchronized with implementation. Lau and Guo's survey of 90 AI coding assistant systems [Lau and Guo, 2025] identifies contextual code understanding as a primary bottleneck; the Documentation Duality standard addresses this by providing pre-structured context at every directory level.

Root `AGENTS.md` defines shared architectural and testing constraints. `.agents/` supplies generic skills, roles, workflows, and confined local state; `.agents/AGENTS.md` documents that catalog. Per-directory `AGENTS.md` adds local contracts, and `README.md` supplies human navigation. `CLAUDE.md` is a client command reference.

3.6 Agentic Skill Architecture

The Documentation Duality standard addresses human and AI navigation at the directory level. A complementary layer operates at the *module* level: capability-specific `SKILL.md` descriptors add discoverable instructions and invocation examples to documented modules. A descriptor supplies context; executable tools remain separately registered interfaces.

3.6.1 The Three-Tier Skill Protocol

`template/` implements a progression of agent-facing documentation, escalating in specificity from global constraints to module-level API contracts:

Tier	File	Scope	Purpose
1 — System	Root `AGENTS.md`, `.agents/`	Repository-wide	Shared constraints, skills, roles, workflows, and local state
2 — Structure	`AGENTS.md`	Documented directories	Local file inventories, API surfaces, integration patterns, architectural constraints
3 — Skill	`SKILL.md`	Skill-enabled modules and exemplars	Machine-parseable skill descriptor: module name, description, key imports, usage pattern

Tier 1 and Tier 2 have direct analogues in the prompt-engineering literature: system prompts and retrieval-augmented context [Lau and Guo, 2025]. Tier 3 is novel. The `SKILL.md` files combine YAML discovery metadata with Markdown instructions. These descriptors complement the separately registered tools exposed through the Model Context Protocol [Anthropic, 2024]. The following presents the metadata from `infrastructure/rendering/SKILL.md`, with its description folded into equivalent YAML for readability:

```yaml
---
name: infrastructure-rendering
description: >-
  Skill for the rendering infrastructure module providing multi-format
  output generation including PDF manuscripts, HTML web pages, and
  Beamer/Reveal.js slides. Use when rendering research outputs,
  converting markdown to PDF, generating slides, or configuring LaTeX
```

```
  rendering.
---
```

A skill loader reading this block discovers the module name and its natural-language activation condition. The Markdown body supplies Python imports and usage examples; the frontmatter does not itself register an executable MCP tool. This is the practical implementation of Toolformer-style self-documented tools [Schick et al., 2023]—rather than a language model learning tool APIs from training data, the APIs are encoded directly in version-controlled, co-located skill files that evolve with the codebase.

3.6.2 Module Skill Coverage

`discover_infrastructure_modules()` reports the actual `README.md`, `AGENTS.md`, `SKILL.md`, and `PAI.md` presence for each enumerated subdirectory. `python -m infrastructure.skills` maintains the shared `.agents/skill_manifest.json`; module and public-exemplar skills participate independently of any editor. `PAI.md` provides an optional integration narrative.

Promotion policy: new Layer-1 directories must ship human + machine-readable docs (`README.md`, `AGENTS.md`) immediately; Tier-3 SKILL assets follow once APIs stabilize.

3.6.3 MCP Server Mapping

The opt-in `infrastructure.mcp_server` exposes the existing operation catalog, pipeline description, discovered skills, and curated CLI invocation through `list_operations`, `describe_pipeline`, `list_skills`, and `invoke_cli`. CLI invocation is confined to registered infrastructure modules; mutating or external operations require the server's explicit opt-in. Skill discovery and tool execution therefore remain separate contracts. The server does not automatically expose every Python function, rendered file, or suggested module-to-resource mapping.

An agent can inspect these catalogs and choose supported operations while retaining the pipeline's declared dependencies and acceptance gates. Discovery alone does not establish successful execution or publication. The `SKILL.md` files parallel Voyager's skill library [Wang et al., 2023]—Voyager's agent accumulates a growing library of executable Minecraft skills represented as JavaScript functions; `template/`'s agent accumulates a curated library of research pipeline skills represented as YAML-frontmattered `SKILL.md` files. In both cases, the skill representation is machine-readable, version-controlled, and self-describing. Wang et al.'s LLM agent survey [Wang et al., 2024a] identifies tool use, planning, and memory as the three fundamental capabilities of autonomous agents; Yao et al.'s ReAct framework [Yao et al., 2023] demonstrates that interleaving reasoning traces with tool actions dramatically improves agent reliability in interactive settings. The `template/` skill architecture maps cleanly onto these three capabilities: the `SKILL.md` descriptors supply the tool-use layer, the declarative DAG of `17 pipeline.yaml` stages (a default full run executes `10`) supplies the planning scaffold, and the per-criterion checkpoint system supplies the memory layer.

3.7 FAIR Alignment and Research Infrastructure as Code

The template's design aligns with both the original FAIR principles [Wilkinson et al., 2016] and the FAIR for Research Software (FAIR4RS) principles [Barker et al., 2022] at the repository level. FAIR4RS recognizes that software has requirements distinct from data—executability, composability, and dependency management—and the template addresses each.

3.7.1 Principle-by-Principle Alignment

Findability. Outputs are *Findable* through standardized directory structures, manifest files, and machine-readable metadata embedded in PDFs. Every project's `config.yaml` provides structured metadata (title, authors, DOIs, keywords) in a format parseable by both Pandoc and external indexing services. The `metrics.json` output provides a machine-readable inventory of all generated artifacts, their locations, and their provenance hashes.

Accessibility. Outputs are *Accessible* via open-source distribution on GitHub, with metadata embedded in the artifact itself rather than in a separate registry. The steganographic layer embeds provenance information directly in the PDF—including SHA-256 content hashes, build timestamps, and QR-encoded metadata—ensuring accessibility even when the PDF circulates outside the repository.

Interoperability. *Interoperability* is achieved through standard formats (PDF, JSON, BibTeX, YAML) and well-defined module APIs that enable cross-project composition. The Pandoc rendering pipeline accepts any Markdown-with-LaTeX input conforming to the template's section numbering conventions, allowing seamless migration of manuscripts from other Pandoc-based workflows.

Reusability. *Reusability* is ensured by the Standalone Project Paradigm—any project can be extracted and reused independently—and by the Documentation Duality standard, which satisfies FAIRsoft's inspectability and documentation quality indicators [Garijo et al., 2024]. The pipeline's automated testing and coverage enforcement directly operationalize the FAIR4RS executability requirement: software that cannot pass its own test suite cannot produce publishable output.

3.7.2 Infrastructure as Code for Research

At a higher level of abstraction, `template/` applies the DevOps principle of *Infrastructure as Code* (IaC) to the research lifecycle. In production software engineering, IaC means that server configuration is version-controlled, automatically provisioned, and independently reproducible [Wilson et al., 2017]. `template/` extends this principle to the research manuscript: the document is not authored in a word processor and emailed to collaborators, but *built* from version-controlled Markdown sources, *tested* against formal coverage thresholds, and *deployed* to a provenance-embedded PDF.

Every component of the research pipeline—the test suite, the analysis scripts, the rendering configuration, and the steganographic watermark—is specified in code, committed to git, and reproducible from a clean checkout. This deterministic build property means that any researcher can clone the repository, run `./run.sh --pipeline`, and produce a byte-for-byte identical manuscript (modulo timestamps in the steganographic metadata).

Software Heritage [Di Cosmo et al., 2020] provides persistent SWHIDs (Software Hash Identifiers) for source code snapshots, enabling stable citation of any specific version of `template/` as a discrete software artifact—closing the loop from research infrastructure to citable scientific contribution. Combined with Zenodo DOI registration (supported by `infrastructure.publishing`), this creates a dual-identifier citation chain: SWHID for source provenance, DOI for publication metadata [Katz et al., 2021].

3.8 Quality Assurance

3.8.1 Zero-Mock Testing Policy

The repository policy prohibits mock frameworks such as `unittest.mock`, `MagicMock`, and `patch` decorators [Martin, 2008, Meszaros, 2007]. A static gate rejects those imports, and a separate inventory surfaces semantic dependency-replacement patterns for review. Tests that require external services (Ollama or public networks) use explicit `pytest.mark` markers for conditional execution; deterministic network tests use real local HTTP servers. The philosophical motivation—analogizing excessive interaction mocking to Simmons et al.'s *researcher degrees of freedom* [Simmons et al., 2011] and the preregistration remedy [Nosek et al., 2018]—is developed fully in the Zero-Mock Tradeoff discussion.

The following example, drawn from this meta-project's own suite, illustrates zero-mock compliance:

```
def test_core_module_present(self):
    # Real filesystem, real YAML parsing - no MagicMock anywhere
    modules = discover_infrastructure_modules(REPO_ROOT)
    names = [m.name for m in modules]
    assert "core" in names, f"'core' not found in {names}"
```

This test (`tests/core/test_meta.py::TestDiscoverInfrastructureModules::test_core_module_present`) exercises the real `discover_infrastructure_modules` function against the real repository root. There are no mock objects substituting for the directory walk, no patched YAML parsers, and no synthetic return values—the test passes only if the infrastructure modules genuinely exist and are discoverable at their expected paths.

3.8.2 Coverage Thresholds

The pipeline enforces two coverage tiers:

Tier	Scope	Minimum	Current	Rationale
Project	`projects/*/src/`	90%	Reported by the current test artifact	Domain code must be thoroughly validated
Infrastructure	`infrastructure/`	60%	Reported by the current test artifact	Broader shared surface

These thresholds are enforced at Stage 01 of the pipeline. A project test run below its declared 90% floor fails before downstream publication stages. This manuscript deliberately does not quote an achieved coverage percentage unless a fresh coverage artifact supplies it.

3.8.3 Test Suite Composition

The repository maintains three test suites:

- **Infrastructure tests** (`tests/`): about 12,548 tests validating the 31 infrastructure subdirectories, covering logging, rendering, validation, steganography, reporting, and LLM integration.
- **Project tests** (`projects/*/tests/`): Per-project suites whose sizes scale with each exemplar's surface area — for example 318 tests in `template_autoresearch_project` and 263 in `template_code_project`, with several exemplars larger still. (A true min/max span would require dedicated `project_test_count_min`/`project_test_count_max` tokens in `build_manuscript_metrics_dict`; see the meta-template's generator backlog.)
- **Integration tests**: Embedded within infrastructure tests, these exercise full pipeline stages against real manuscript inputs, validating end-to-end behavior from Markdown source to rendered PDF.

3.8.4 Visualization Standards

All generated figures must meet accessibility requirements:

- Shared 16pt font constant for primary text elements, with compact annotations derived from that constant.

- Colorblind-safe palettes (IBM Design / Wong palette) with high-contrast fallbacks.
- 200 DPI PNG rendering, sourced from the same constant used by every figure writer.
- Descriptive axis labels and figure titles.
- No reliance on color alone to convey information—redundant encoding via shape, pattern, or annotation is used where applicable.

The `tests/figures/test_architecture_viz.py` suite verifies that each real figure generator writes a non-empty PNG and that the comparative matrix has its declared shape and value domain. The font and render-resolution values above are injected from `src/template_template/figures/viz_palette.py`, preventing prose and implementation from drifting even though visual accessibility still requires human inspection of rendered figures.

4 Results

This section reports repository facts produced by the meta-project's live introspection pass. Test counts, module inventories, pipeline-stage counts, and figure data are injected from generated artifacts. Runtime and achieved-coverage claims are intentionally omitted when no versioned benchmark or coverage artifact backs them.

4.1 Multi-Project Pipeline Execution

The `./run.sh` interactive orchestrator can execute the public roster through the shared DAG. The table below is a structural snapshot, not a timing benchmark: counts come from repository introspection, and the coverage column states each exemplar's declared policy floor rather than an unversioned observed percentage.

Project	Effective core stages[1]	Discovered tests	Declared project floor
`template_code_project`	8	263	90%
`template_prose_project`	8	165	90%
`template_autoresearch_project`	8	318	90%

[1]"Core-only" excludes LLM-tagged and other opt-in stages according to the YAML stage tags. A fresh run must be used to establish completion status or wall-clock performance for a particular machine and dependency set.

4.2 Infrastructure Test Suite

Metric	Value
Test files	948+
Total tests	about 12,548
Infrastructure coverage gate	>=60% configured floor
Prohibited mock-framework imports	Checked by the static no-mocks gate

Exercises Pandoc/XeLaTeX paths, steganography hashing, telemetry, YAML-driven pipeline DAG resolution, HTTPS-bound optional suites (`pytest-httpserver`), and local Ollama-gated subsets.

4.3 Infrastructure Module Inventory

The introspection module (`template_template.core.introspection`) emits the authoritative table below—every row reflects `discover_infrastructure_modules(REPO_ROOT)`.

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
`autoresearch`	11	✓	✓	`build_autoresearch_plan`, readiness validation CLI
`benchmark`	4	✓	✓	Template harness scoring + comparative gates

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
`config`	0	✓	✓	Repository defaults + hardened templates
`core`	201	✓	✓	`get_logger`, `load_config`, `TemplateError`
`docker`	0	✓	✓	Containerisation scaffolding
`doctor`	14	✓	✓	Checkout diagnose/fix/undo repairs
`documentation`	23	✓	✓	`FigureManager`, `build_api_index`, `generate_markdown_table`
`fonds`	6	✓	✓	—
`formal_contracts`	5	✓	✓	—
`llm`	57	✓	✓	Ollama helpers, sanitization, review + translation pipelines
`logrotate.d`	0	✓	✓	Rotation snippets (documentation-first)
`metadata`	3	✓	✓	`normalized_repository_url`, `generate_metadata_package`
`methods`	10	✓	✓	`build_methods_orchestration_plan`, methods-stage contracts + validation
`orchestration`	14	✓	✓	`PipelineRunner`, entry point for `./run.sh`
`project`	116	✓	✓	`discover_projects`, workspace management
`prose`	9	✓	✓	Markdown readability + prose tooling
`provenance`	9	✓	✓	—
`publishing`	85	✓	✓	Zenodo, executable bundle, archival targets
`reference`	16	✓	✓	BibTeX models, parsers, converters
`rendering`	120	✓	✓	PDF/HTML/slide rendering, Pandoc filters

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
`reporting`	411	✓	✓	Coverage parsers, dashboards, executive artefacts
`research`	3	✓	✓	—
`rules`	6	✓	✓	—
`scientific`	4	✓	✓	`check_numerical_stability`, `benchmark_function`
`search`	77	✓	✓	`infrastructure.search.literature` clients + cache
`sia`	10	✓	✓	Self-Improving-AI loop: task validation, harness, metric capture
`skills`	8	✓	✓	`discover_skills`, SKILL manifest regeneration
`steganography`	13	✓	✓	Watermark overlays + hash manifests
`tools`	6	✓	✓	—
`transmission`	6	✓	✓	—
`validation`	87	✓	✓	PDF + Markdown + integrity CLIs

All 31 enumerated subdirectories carry Tier-1/`README.md` and Tier-2/`AGENTS.md` coverage wherever the Documentation Duality standard applies; subsets ship Tier-3 `SKILL.md` descriptors for MCP routing (`infrastructure/skills` manifest generation).

4.4 Agentic Skill Documentation Coverage

Layer	Role
Shared agent guidance	Root `AGENTS.md`, `.agents/` skills, roles, and workflows
Structural	`AGENTS.md` directories
Skills	Optional `SKILL.md` manifests + generated `.agents/skill_manifest.json`
PAI capsule	Repository level `PAI.md` narratives

`379+` Markdown shards under `docs/` capture operational knowledge without duplicating auto-generated inventories.

4.5 DAG Reference (Declarative Executor)

The table summarizes the 10 default full-run stages (core plus live LLM nodes) from 17 YAML declarations; core mode selects 8. Separately tagged opt-in extensions are omitted here. Entries follow dependency order rather than numeric script filenames.

Name	Typical script / method	Responsibility	Failure semantics
Clean Output Directories	`_run_clean_outputs`	Deletes stale `output/` trees	Blocking
Environment Setup	`scripts/pipeline/stage_00_setup.py`	Validates tooling, PYTHONPATH scaffolding	Blocking
Infrastructure Tests	`scripts/pipeline/stage_01_test.py --infra-only --infra-scope pipeline-smoke`	Focused infrastructure smoke contracts	Required tests must pass
Project Tests	`scripts/pipeline/stage_01_test.py --project-only`	Project-local tests + coverage gates	Required tests and coverage must pass
Project Analysis	`scripts/pipeline/stage_02_analysis.py`	Executes `projects/<name>/scripts/*.py`	Blocking
PDF Rendering	`scripts/pipeline/stage_03_render.py`	Pandoc → XeLaTeX manuscripts	Blocking
Output Validation	`scripts/pipeline/stage_04_validate.py`	Structural PDF/markdown probes	Blocking / warnings
LLM Manuscript Review	`scripts/pipeline/stage_06_llm_review.py --reviews-only`	Local Ollama reviews	Fresh policy-bound skip; required live acceptance remains blocking
LLM Translations	`scripts/pipeline/stage_06_llm_review.py --translations-only`	Optional translations	Fresh policy-bound skip
Copy Outputs	`scripts/pipeline/stage_05_copy.py`	Mirrors deliverables → `output/<project>/`	Blocking

`scripts/pipeline/stage_07_executive_report.py` provides separate **multi-project executive reporting**, not an additional single-project DAG node (`execute_pipeline.py`). The multi-project executor writes its aggregate summary after project execution.

4.6 Steganographic Performance Boundary

The steganography path exposes metadata injection, SHA-256 hashing, overlay generation, and optional QR-code insertion as separately testable operations. This revision makes no latency claim because the project does not currently generate a versioned benchmark artifact recording hardware, input PDF, warm-up policy, repetitions, and dispersion. A future performance table should be compiled from such an artifact rather than copied from an ad-hoc workstation run.

4.7 Self-Referential Analysis

Rendered via `projects/templates/template_template` (`generate_manuscript_metrics.py` → injected tokens such as `31`). Architecture figures stem from `template_template.figures.architecture_viz`—font sizes constrained by §QA.

The long tail of small, single-purpose packages beside a handful of larger ones (`core`, `validation`, `publishing`) is the visual signature of the Unix-philosophy modularity the architecture section argues for—capability concentrated where it compounds, not spread evenly by fiat.

4.8 Comparative Feature Analysis

The comparative feature matrix below summarizes the Appendix F capability matrix.

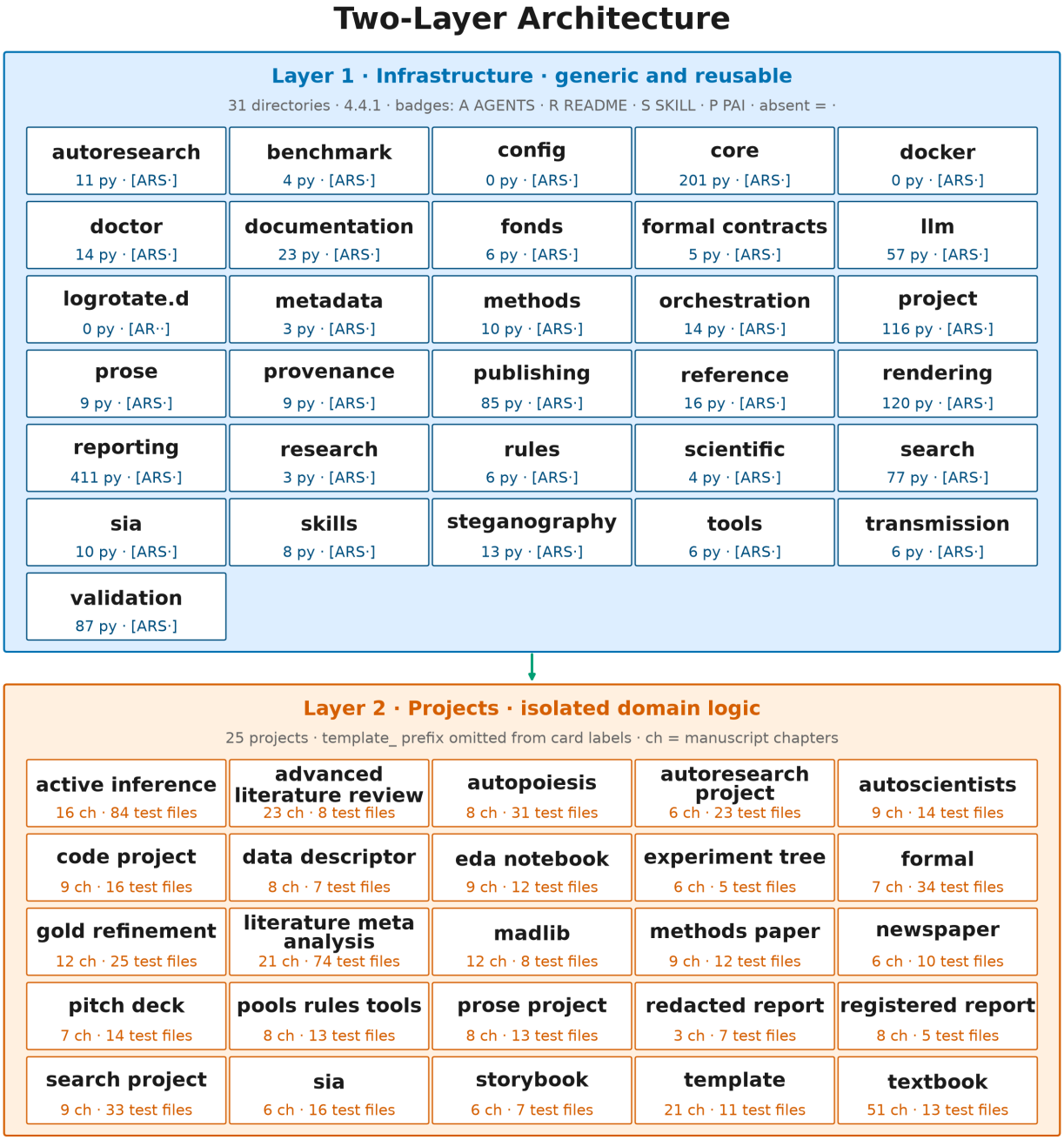


Figure 3: Live rendering of the Two-Layer Architecture from repository introspection: the infrastructure layer (top) holds the `31` enumerated infrastructure directories, each annotated with its Python file count and a four-slot documentation badge—`A` AGENTS.md, `R` README.md, `S` SKILL.md, `P` PAI.md, with · marking an absent file—so a fully documented module reads `[ARSP]`. A YAML DAG arrow connects it to the project layer (bottom) of public exemplars labelled with chapter and test-file counts. The takeaway: documentation-duality coverage is near-uniform across the infrastructure, and every box was placed from the same live data the prose cites.

Stages 1–5 of 17

1	Clean Output Directories	core clean
2	Environment Setup	core
3	Infrastructure Tests	core tests
4	Project Tests	core tests
5	Project Analysis	core

Stages 6–10 of 17

6	Connector Search	science live
7	Provenance Record	provenance
8	PDF Rendering	core
9	Output Validation	core
10	LLM Manuscript Review	llm live

Stages 11–15 of 17

11	LLM Translations	llm live
12	Copy Outputs	core
13	Ebook Generation	core ebook
14	docxplus Export	core docxplus
15	Metadata Package	core metadata

Stages 16–17 of 17

16	Executable Bundle	bundle
17	Archival Publication	archival live

Figure 4: Declared-stage reference with all 17 YAML stages in reading order, showing complete names and tags (core, LLM, ebook, docxplus, metadata, bundle, archival). Panel order documents declarations; it does not imply a linear dependency chain.

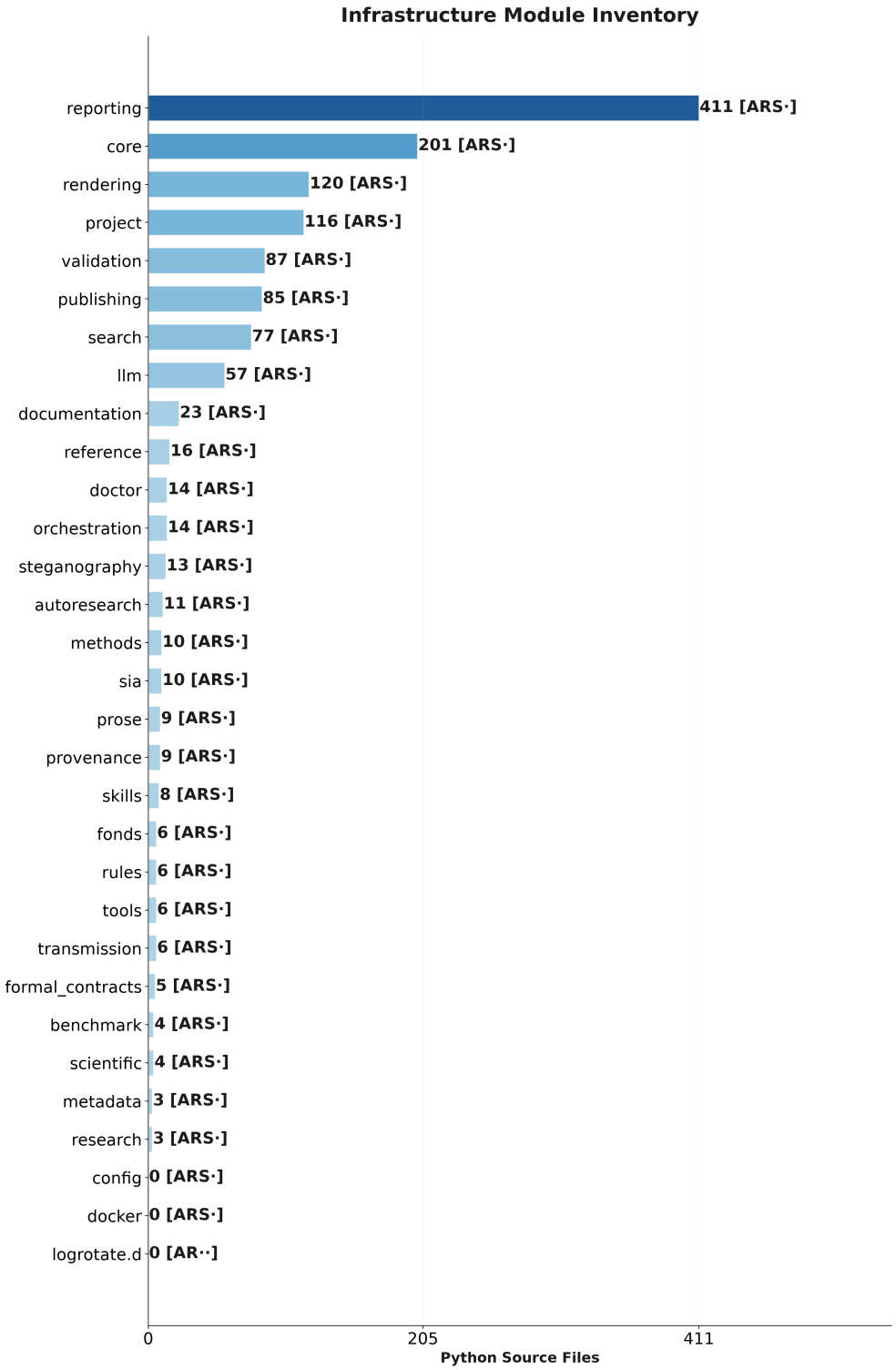


Figure 5: Horizontal file-count histogram of every infrastructure subdirectory, sorted largest-first.



Figure 6: 14×10 heatmap annotated in appendix text—green ✓ full native capability, yellow partial / plugin-hosted, red — unavailable. Rows group *Core Pipeline*, *Quality & Security*, then *Ecosystem*.

[1] Nextflow lineage records exist at workflow scope, not per rendered PDF citation graph.
[2] DVC: content-addressed artifacts without native prose rendering.
[3] DVC: remote object stores (S3, GCS, Azure) without turnkey CI manuscript gates.

4.9 Test Quality Metrics

- **Zero mocks:** repository policy bans `unittest.mock` / patching frameworks in tests.
- **Filesystem + subprocess realism:** ephemeral directories + actual CLI binaries.
- **HTTP realism:** infra suites favour `pytest-httpserver`; literature tests hit recorded fixtures.
- **`template_code_project`** focuses on numerical reproducibility assertions.
- **`template_autoresearch_project`** exercises the AutoResearch readiness planner (`infrastructure/autoresearch/`). **`template_search_project`** exercises literature-search workflows.

5 Discussion

5.1 The Zero-Mock Tradeoff

The Zero-Mock testing policy is `template/`'s most distinctive design decision. By prohibiting all mock objects, we gain confidence that tests exercise real code paths—a pytest run against the template genuinely invokes `pandoc`, writes to disk, and parses real YAML. The cost is test duration: the full infrastructure test suite (about 12,548 tests) runs for minutes rather than the sub-second execution typical of heavily-mocked suites. This manuscript deliberately reports no wall-clock figure, consistent with the Results section's discipline of declining timing claims without a versioned benchmark artifact.

We argue this tradeoff is strongly favorable for research software. Unlike web applications where millisecond latency and thousands of daily deploys demand fast feedback loops, research pipelines run infrequently (once per manuscript revision) and correctness vastly outweighs speed. A mocked test that passes while the real renderer fails is worse than a slow test that catches the failure. The analogy to statistical methodology is precise: just as Simmons et al.'s *researcher degrees of freedom* [Simmons et al., 2011] inflate false-positive rates through undisclosed analytical flexibility, mock objects create *testing degrees of freedom* that make integration failures invisible. The Zero-Mock policy closes this loophole by the same mechanism that pre-registration [Nosek et al., 2018] closes the p-hacking loophole: removing flexibility before the fact. As Peng [Peng, 2011] argues, computational reproducibility requires independent verification—and mock-only tests verify assumptions rather than results. Garijo et al.'s FAIRsoft evaluator [Garijo et al., 2024] identifies *executability* as a primary quality indicator; the Zero-Mock policy operationalizes executability at the unit level.

5.1.1 When Mocks Are Not the Problem

It is important to distinguish the Zero-Mock policy from a naive rejection of all test isolation techniques. Fowler's classification [Martin, 2008] recognizes that stubs and fakes serve legitimate purposes—a test database populated with known data is not a mock, it is a fixture. The policy specifically prohibits *mock objects* as defined by Meszaros: assertions on indirect outputs (method calls, argument patterns) rather than direct outputs (return values, side effects). The distinction matters because mock-based assertions encode implementation assumptions ("method X must be called with argument Y") that become invisible coupling between tests and production code, creating the illusion of coverage without testing real behavior.

5.1.2 Practical Implementation

The template enforces zero-mock compliance at three levels:

1. **Code review**: `AGENTS.md` at every directory level explicitly states the prohibition, ensuring both human and AI contributors are aware before writing tests.
2. **Static analysis**: `scripts/audit/verify_no_mocks.py` implements the enforced lexical gate (prohibited mock-framework imports and calls) and an `--inventory` mode that separately classifies environment isolation from dependency-replacement debt; pre-push runs it automatically.
3. **Cultural norm**: `template_code_project` documents filesystem + YAML + plotting paths while `template_autoresearch_project` exercises readiness planning; `template_search_project` reinforces HTTP-realistic literature queries—both serve as onboarding references alongside infra suites.

However, the policy requires careful management of external dependencies. Tests requiring Ollama (the local LLM backend) use `@pytest.mark.requires_ollama` and are skipped in environments where the service is unavailable. Tests requiring network access use `@pytest.mark.network`. This marker system preserves the Zero-Mock principle while acknowledging that not all environments provide all services, especially computationally intensive ones. The key distinction is between *replacing* an external dependency (which mock objects do, hiding failures) and *skipping* a test when a dependency is absent (which markers do, preserving transparency).

5.2 Scalability: From 1 to N Projects

The Standalone Project Paradigm enables horizontal scaling: a project supplies its manuscript/config, isolated source and tests, thin analysis scripts, and declared dependencies. New projects use existing infrastructure interfaces; admission to the tracked public roster and release matrix remains explicit. The `run.sh` orchestrator automatically discovers new projects and presents them in its interactive menu.

We have validated scaling with 25 canonical exemplars under `projects/templates/`—always present for onboarding and tooling—and with this manuscript from `projects/templates/template_template` (158 tests) as a git-tracked public exemplar in the same automated discovery menus.

Canonical trio:

- **`template_code_project`**: Numerical optimization example with gradient-descent narration and 263 discovered tests. Its `pyproject.toml` declares the 90% project-source floor. Minimal footprint: compact `src/`, scripted analysis, short manuscript sections.
- **`template_prose_project`**: Prose-heavy manuscript emphasizing narrative structure and bibliography discipline, with 165 discovered tests and the same declared project-source floor; tests exercise rendering and Markdown integrity without heavyweight numerics.
- **`template_autoresearch_project`**: AutoResearch readiness workflow — bounded ML task loop (deterministic MNIST run), plan validation (`infrastructure.autoresearch`), evidence-registry reports, and manuscript hydration; its own scripts (`run_autoresearch_loop.py`, `z_generate_manuscript_variables.py`) stay thin over `src/` logic. **`template_search_project`** exercises literature-search workflows (remote corpus JSON fusion, `y_generate_search_figures.py`, review reports).

Meta manuscript (**`projects/templates/template_template`**) analyzes the repository via `src/template_template/` introspection metrics; it now lives alongside the other public exemplars under `projects/templates/`.

These workspaces share no project-level code—only Layer 1 (31 infrastructure subdirectories, about 1334 Python files)—validating insulation between domain repos and reusable services.

5.2.1 Multi-Project Orchestration

When the `--all-projects` flag is passed to `run.sh`, the pipeline executes each discovered project sequentially, running infrastructure tests once at the start and skipping them for individual projects to avoid redundant validation. After all projects complete, a cross-project executive report aggregates metrics (test counts, coverage percentages, page counts, rendering durations) into a unified dashboard with both JSON and Markdown output formats. This executive reporting stage provides repository-level visibility without requiring any project-specific reporting code.

5.2.2 Scaling Metrics

Metric	`template_code_project`	`template_prose_project`	`template_autoresearch_project`
Source modules	25	8	59
Test files	16	13	23
Test count	263	165	318
Manuscript chapters	9	8	6
Analysis scripts	9	5	5
Figures (auto-generated)	9	5	27

The infrastructure overhead per project is constant regardless of project size: the same 31 modules, the same

17 declared pipeline stages, the same rendering and validation logic. The shared implementation does not grow with the project roster; execution time and output volume still grow with the work performed.

5.3 Comparison to Existing Tools

The gap analysis established that no single tool integrates all six cross-cutting concerns. Here we synthesize the fourteen-dimension comparison into three structural insights. First, the landscape bifurcates: workflow managers (Snakemake [Köster and Rahmann, 2012], Nextflow [Di Tommaso et al., 2017], CWL [Amstutz et al., 2016]) provide distributed execution but no manuscript support; publication tools (Quarto [Allaire et al., 2024], Jupyter Book, R Markdown [Xie et al., 2018], Overleaf [Overleaf (Digital Science), 2025], Prism [OpenAI, 2026]) author documents but embed no integrity guarantees; and DVC [Iterative, Inc., 2024] versions artifacts without orchestrating pipelines. `template/` occupies the intersection, sacrificing distributed execution for unified enforcement of testing, provenance, and documentation. This positioning is complementary—a mature deployment might use Nextflow upstream and `template/` for rendering, testing enforcement, and provenance downstream. Typst [Mädje and Haug, 2023], with its faster compilation cycle, is not one of the nine compared peers but could serve as an alternative rendering backend if a Pandoc writer were contributed.

Second, the eight enforcement capabilities `template/` co-locates—testing enforcement, coverage thresholds, steganographic watermarking, multi-project management, AI-agent documentation, the agentic skill protocol, an interactive TUI, and Zero-Mock policy—are individually straightforward (and several, such as multi-project management and AI-agent documentation, are matched in part by individual peers). Their value here lies in a shared pipeline contract: required test and documentation gates run together, while selected provenance stages carry their own validation. The FAIR4RS principles [Barker et al., 2022, Lamprecht et al., 2020] articulate what research software quality requires; FAIRsoft [Garijo et al., 2024] scores compliance observationally; `template/` operationalizes part of this agenda by failing project tests below the declared 90% floor and validating provenance artifacts when those stages are enabled. Cohen et al.'s four pillars of research software engineering [Cohen et al., 2021]—sustainability, quality, community, and policy—are addressed primarily through the first two pillars via quality-gated automation.

Third, the AI-agent documentation dimension reveals an underserved need. Overleaf and Prism provide AI *writing* assistance, but neither exposes structured documentation for *external* agents to consume. `template/`'s `AGENTS.md` + `SKILL.md` layer enables an agent entering the repository to discover capabilities, understand API contracts, and invoke modules without prior training (Documentation Duality, AI Collaboration).

5.3.1 FAIR4RS Evolution (2024–2026)

Since the FAIR4RS principles were published [Barker et al., 2022], the community has moved toward operationalization. The RDA Virtual Plenary 24 (April 2025) featured a two-year retrospective review [Honeyman et al., 2024] recommending principle amendments—notably adding *reproducibility* as an explicit requirement and clarifying "domain-relevant standards"—alongside a leadership refresh and parallel guidance activities. The ReSA Actionable FAIR4RS Task Force (launched December 2024) analyzed the 17 principles into six actionable categories (identifiers, metadata for publication/discovery/reuse, standards, references, and licenses), with a first draft expected by September 2025 [Research Software Alliance, 2024]. Tools for automated FAIR assessment have also matured: the F-UJI extension for research software evaluation now scores against FRSM-04 through FRSM-17 metrics, complementing Garijo et al.'s FAIRsoft evaluator [Garijo et al., 2024]. `template/`'s pipeline-enforced quality gates—coverage thresholds, documentation completeness checks, and provenance embedding—anticipate this operationalization trend by implementing FAIR4RS not as a post-hoc assessment but as an architectural invariant.

In Gentleman and Temple Lang's terminology [Gentleman and Temple Lang, 2007], `template/` is a *research compendium* scaled to the repository level—bundling not just one study's code and data but N studies, with shared infrastructure, automated testing, and embedded provenance. Nüst et al.'s executable research compendium (ERC) [Nüst et al., 2017] extends this vision with containerized reproduction environments; `template/` complements containerization by adding the testing enforcement, multi-project management, and provenance embedding layers that ERCs do not address.

5.4 The AI Collaboration Model

The Documentation Duality standard and three-tier skill architecture represent an empirical bet: that structured, machine-readable documentation measurably improves AI agent performance in research codebases. This section reports our key observations.

The documentation investment creates a positive feedback loop: as agents produce higher-quality outputs from structured context, developers maintain that documentation, which in turn improves future interactions [Lau and Guo, 2025]. We observed this concretely during `template/` development—capability-specific `SKILL.md` descriptors were refined through iterative AI-assisted generation, serving as both input prompt and output validator.

The `SKILL.md` layer, with its MCP-aligned YAML frontmatter [Anthropic, 2024], provides a bridge to the agentic software paradigm. Lu et al.'s AI Scientist [Lu et al., 2024] demonstrates end-to-end autonomous research, while OpenHands is evaluated on SWE-Bench Verified [Wang et al., 2024b, Jimenez et al., 2024]. These systems motivate structured, protocol-aligned tool inventories for unfamiliar codebases. An agent navigating `template/` reads root `AGENTS.md` and `.agents/AGENTS.md` for shared constraints and generic workflows, scans `AGENTS.md` for local contracts, and can invoke skill-enabled capabilities through `SKILL.md` descriptors. The live module inventory verifies `AGENTS.md` and `README.md` coverage; `SKILL.md` remains a capability-specific layer rather than a universal claim about every infrastructure directory.

This three-tier model is, to our knowledge, novel in the research software engineering literature. The scale of the investment is substantial: `379` Markdown files under `docs/` alone, plus an `AGENTS.md`/`README.md` pair in documented directories and a `SKILL.md` descriptor on skill-enabled infrastructure modules. That count is itself injected from live introspection—the manuscript refuses to quote a documentation total it cannot recompute—and it represents a deliberate commitment to machine-readable context that shrinks the surface on which an agent can hallucinate.

5.5 The Learning Curve

The Thin Orchestrator pattern imposes a cognitive overhead on researchers accustomed to writing monolithic scripts. The requirement to factor all logic into `src/` modules and use scripts only as stateless wiring introduces an additional layer of indirection. We mitigate this through:

1. **Template exemplars**: `template_code_project` ships minimal optimization commentary; `template_prose_project` and `template_autoresearch_project` broaden narrative + retrieval scaffolding.
2. **Documentation Duality**: Every directory has both `README.md` (for humans) and `AGENTS.md` (for AI collaborators), reducing the cost of navigation.
3. **Interactive orchestrator**: `run.sh` provides a TUI menu that abstracts pipeline complexity.
4. **Skill-level documentation**: The `docs/guides/` directory provides a progressive sequence of beginner, intermediate, advanced, and expert guides alongside a comprehensive new-project setup checklist.

5.6 Limitations

- **LaTeX dependency**: The rendering pipeline requires a full TeX distribution (TeX Live or MiKTeX), which is a 4–6 GB install. This is the largest single dependency and is a barrier for researchers without system-level package management access.
- **Python-centric**: The infrastructure layer is Python-only. Projects in other languages can use the rendering and steganography stages but cannot leverage the `scientific` or `validation` modules.
- **Single-machine**: The pipeline runs locally. Distributed execution (e.g., across a compute cluster) is not natively supported, a gap where Snakemake, Nextflow, and CWL have clear superiority.
- **Steganographic robustness**: Alpha-channel overlays are stripped by aggressive PDF optimization tools (e.g., `qpdf --optimize`). QR codes are visible and removable. The current system provides *tamper detection* (via SHA-256 hashing) but not *non-repudiation* in the cryptographic sense—it lacks private-key digital signatures. An attacker with access to the source code could reproduce the watermark without having run the original pipeline.
- **Test duration**: The Zero-Mock policy increases test execution time from sub-second (mocked) to multi-minute (real) for the full infrastructure suite. This is acceptable for research workflows but may not suit continuous deployment scenarios.

- **AI-native writing tools**: `template/` does not include an integrated AI writing assistant comparable to Overleaf's Copilot features or OpenAI Prism's GPT-5.2 context-aware editing. The `infrastructure.llm` module provides LLM review as a pipeline stage but not as an interactive writing environment.

5.7 Future Directions

1. **Supply-chain provenance**: Integration with software supply chain frameworks such as in-toto [Torres-Arias et al., 2019] and SLSA (Supply-chain Levels for Software Artifacts) [Open Source Security Foundation, 2023] to provide end-to-end attestation from source commit through build pipeline to published artifact. SLSA's four graduated levels of build integrity (from basic provenance to hermetic, reproducible builds) provide a natural extension ladder for the template's currently document-centric provenance model. The template's existing steganographic layer embeds document-level provenance; supply-chain frameworks would add build-level provenance, closing the gap between "this PDF was produced by this pipeline" and "this pipeline was executed with this verified source code."
2. **Decentralized provenance**: Integration with IPFS or Arweave for immutable publication records, extending the SHA-256-based tamper detection to content-addressed storage networks.
3. **Digital signatures**: GPG or X.509 signing integrated into the steganographic layer, providing cryptographic non-repudiation in addition to tamper detection.
4. **Continuous integration**: GitHub Actions workflows that execute the pipeline on every push, with PDF artifacts as release assets and automated DOI registration via Zenodo.
5. **Multi-language support**: Extension of the Thin Orchestrator pattern to R, Julia, and Rust projects, enabling polyglot research workflows within the Two-Layer Architecture.
6. **Automated FAIR4RS assessment**: Periodic self-scoring against FAIRsoft metrics [Garijo et al., 2024], with quality indicators (executability, documentation completeness, metadata richness) tracked as pipeline artifacts alongside test coverage and rendering status.
7. **Knowledge graph integration**: Connecting pipeline outputs to Active Inference Knowledge Base entries for automated meta-analysis and cross-project citation tracking.
8. **Formal verification**: Static analysis tooling to enforce the Thin Orchestrator pattern—verifying that scripts contain no algorithmic logic and that `src/` modules do not import from `scripts/`.
9. **Agentic research pipelines via MCP**: The `SKILL.md` descriptors already define the interface contracts for each infrastructure module; the natural next step is to expose them as MCP server endpoints [Anthropic, 2024]. An MCP server wrapping `infrastructure.llm` would expose `query`, `review_manuscript`, and `translate_abstract` as protocol-native Tools; an MCP server wrapping `infrastructure.publishing` would expose `publish_to_zenodo` and `generate_citation_bibtex`. A research agent could then compose these Tools to execute the full pipeline—environment setup → test execution → analysis → rendering → validation → LLM review → DOI registration—without any human in the loop. This closes the loop opened by Lu et al.'s AI Scientist [Lu et al., 2024], which demonstrated automated hypothesis generation and experimental iteration but relied on ad hoc laboratory scaffolding. `template/`'s pipeline, fully exposed as MCP tools, provides that scaffolding in a reproducible, versioned, and certified form. Longer-term, the Agent2Agent (A2A) protocol [Google, 2025] enables heterogeneous specialist agents—a statistical analyst, a figure designer, a peer-review simulator—to coordinate via a shared, protocol-mediated workspace built on precisely the kind of modular, well-documented infrastructure that `template/` provides.
10. **Research Infrastructure as Code and software citation**: The DevOps IaC paradigm, applied to research, means the entire manuscript pipeline is a version-controlled, reproducible artifact in its own right. Software Heritage [Di Cosmo et al., 2020] provides persistent SWHIDs (Software Hash Identifiers) for source-code snapshots, enabling `template/` itself to be cited as a software artifact with a DOI-equivalent stable identifier. Combining this with Zenodo DOI registration (already supported by `infrastructure.publishing`) creates a full citation chain: the paper cites the data (DOI), the data provenance cites the pipeline (SWHID), and the pipeline cites the framework release (Zenodo DOI). This three-link citation chain operationalizes the Katz et al. [Katz et al., 2021] software citation principles at the infrastructure level.

5.8 Conclusion

`template/` demonstrates that high-integrity, reproducible research need not be onerous. By embedding provenance, testing, and documentation into the architecture itself—rather than layering them atop a fragmented workflow—the template transforms "best practices" from aspirational guidelines into enforced invariants [Wilson et al., 2017, Sandve et al., 2013, Lamprecht et al., 2020]. The Two-Layer Architecture ensures that

infrastructure improvements propagate to all projects without coupling. The Zero-Mock policy ensures that tests reflect reality. The steganographic provenance layer ensures that published artifacts carry their own authentication. The comparative analysis distinguishes native, partial, and absent support across its versioned capability inventory. The total comparison inventory includes capabilities that `template/` does not fully support, such as distributed execution and multi-language research. It is not the count of fully native `template/` capabilities, and the snapshot does not establish an exhaustive claim about all existing tools.

The template is not merely a build tool; it is an epistemological commitment. It asserts that a research paper is not a static document but a build artifact—reproducible, verifiable, and traceable to the code that generated it. As Knuth observed, programs should be written for humans to read and only incidentally for machines to execute [Knuth, 1984]. We extend this dictum: research manuscripts should be *built* for verification and only incidentally for reading. In an era of generative AI, AI-native research workspaces, and synthetic media—where the boundary between human-authored and machine-generated text grows increasingly indeterminate [Gruenpeter et al., 2021]—the provenance chain from source code to published PDF is not an administrative convenience. It is the epistemic ground on which scientific trust must be rebuilt. That this manuscript was itself built, tested, and watermarked by the pipeline it describes—its metrics computed from the repository it inhabits, its figures rendered by the code it documents—demonstrates one reproducible artifact path; broader reliability claims require the declared checks and candidate-bound execution evidence.

6 Infrastructure Module Reference

This section inventories every Layer-1 subdirectory returned by `31 discover_infrastructure_modules(repo_root)`. File totals use `1334` Python sources across infra + `12,548` infra tests guarding them. Documentation Duality = paired `README.md` + `AGENTS.md`; optional `SKILL.md` manifests feed `python -m infrastructure.skills`.

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
`autoresearch`	11	✓	✓	`build_autoresearch_plan`, readiness validation CLI
`benchmark`	4	✓	✓	Template harness scoring + comparative gates
`config`	0	✓	✓	Repository defaults + hardened templates
`core`	201	✓	✓	`get_logger`, `load_config`, `TemplateError`
`docker`	0	✓	✓	Containerisation scaffolding
`doctor`	14	✓	✓	Checkout diagnose/fix/undo repairs
`documentation`	23	✓	✓	`FigureManager`, `build_api_index`, `generate_markdown_table`
`fonds`	6	✓	✓	—
`formal_contracts`	5	✓	✓	—
`llm`	57	✓	✓	Ollama helpers, sanitization, review + translation pipelines
`logrotate.d`	0	✓	✓	Rotation snippets (documentation-first)
`metadata`	3	✓	✓	`normalized_repository_url`, `generate_metadata_package`
`methods`	10	✓	✓	`build_methods_orchestration_plan`, methods-stage contracts + validation
`orchestration`	14	✓	✓	`PipelineRunner`, entry point for `./run.sh`
`project`	116	✓	✓	`discover_projects`, workspace management

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
`prose`	9	✓	✓	Markdown readability + prose tooling
`provenance`	9	✓	✓	—
`publishing`	85	✓	✓	Zenodo, executable bundle, archival targets
`reference`	16	✓	✓	BibTeX models, parsers, converters
`rendering`	120	✓	✓	PDF/HTML/slide rendering, Pandoc filters
`reporting`	411	✓	✓	Coverage parsers, dashboards, executive artefacts
`research`	3	✓	✓	—
`rules`	6	✓	✓	—
`scientific`	4	✓	✓	`check_numerical_stability`, `benchmark_function`
`search`	77	✓	✓	`infrastructure.search.literature` clients + cache
`sia`	10	✓	✓	Self-Improving-AI loop: task validation, harness, metric capture
`skills`	8	✓	✓	`discover_skills`, SKILL manifest regeneration
`steganography`	13	✓	✓	Watermark overlays + hash manifests
`tools`	6	✓	✓	—
`transmission`	6	✓	✓	—
`validation`	87	✓	✓	PDF + Markdown + integrity CLIs

6.1 Alphabetical summaries

Below, `${module_*_python_file_count}` placeholders expand per subdirectory at render-time.

6.1.1 `infrastructure.autoresearch` (11 files)

Readiness planner, validation CLI, and report models for AutoResearch-style project promotion (`infrastructure/autoresearch/`).

6.1.2 `infrastructure.benchmark` (4 files)

Template harness scoring and comparative gate helpers exercised in CI smoke paths.

6.1.3 `infrastructure/config` (non-package subdirectory)

Repository-wide YAML templates and secure manifests (`.env.template`, hardened defaults referenced by Docker + CLI). `config/` carries no `__init__.py`, so it is a configuration subdirectory rather than an importable package.

6.1.4 `infrastructure.core` (201 files)

Checkpointing, logging, pipeline YAML parsing, telemetry bridges, filesystem helpers, hardened exceptions. Everything else imports logging + error taxonomy from here first.

6.1.5 `infrastructure.doctor` (14 files)

Checkout diagnose/fix/undo repairs for broken local workspace states.

6.1.6 `infrastructure.docker` (0 files)

Pinned images / compose scaffolding for reproducible CI + remote builds.

6.1.7 `infrastructure.documentation` (23 files)

Figure registries plus glossary tooling feeding manuscript automation.

6.1.8 `infrastructure.formal_contracts` (5 files)

Typed, machine-checkable manuscript contract layer: frozen block model, evidence-tier policies, monoidal composition, and a fail-closed checker with stable `FORMAL.*` diagnostics.

6.1.9 `infrastructure.fonds` (6 files)

Resource pool management for curated fonds (tracked reference datasets, bibliographic collections, and evidence corpora). Fonds mirror `projects/templates/` with git-tracked `templates/*` exemplars and sidecar-linked private lifecycle folders.

6.1.10 `infrastructure.llm` (57 files)

Ollama integrations, sanitization adapters, templated reviewer flows. **Literature ingestion now lives primarily in `search/literature` + citation helpers in `reference/`.**

6.1.11 `infrastructure.metadata` (3 files)

Shared publication-metadata leaves: repository-URL normalization (`normalized_repository_url`) and ebook metadata package generation (ONIX, `metadata.json`, EPUB OPF). Consumed by publishing, rendering, documentation, and project layers; extracted from `publishing` by RENDERING-LAYERING-1 so renderers import it without an inverted publishing dependency.

6.1.12 `infrastructure.methods` (10 files)

Deterministic methods-orchestration contracts (`MethodStage`, `MethodsOrchestrationPlan`, `MethodsIssue`): builds and validates an ordered methods plan for a research project so the manuscript's "Methods" track stays bound to executable stages.

6.1.13 `infrastructure.orchestration` (14 files)

`python -m infrastructure.orchestration` exposes interactive menus, subprocess wiring for thin shell wrappers (`run.sh`, `secure_run.sh`), and stubs used in CI for menu parsing tests.

6.1.14 `infrastructure.project` (116 files)

Canonical discovery (`discover_projects`) enforcing `src/` + `tests/`, slug validation, nested WIP namespaces.

6.1.15 `infrastructure.prose` (9 files)

Readability metrics + Markdown tooling for prose-centric manuscripts / CI gates.

6.1.16 `infrastructure.provenance` (9 files)

Content-addressed provenance DAG. Records artifact lineage (which run produced which file, from which inputs) as a verifiable graph of artifact/run/source/claim nodes connected by produced/consumed/derived-from/supports/refutes edges. Includes a structured Review system with severity (blocking/major/minor/info) and verdict (refutes/supports). Features a CLI and pipeline integration hooks for automatic lineage recording after every stage.

6.1.17 `infrastructure.publishing` (85 files)

Metadata models, APA/BibTeX/MLA formatters, optional Zenodo clients.

6.1.18 `infrastructure.reference` (16 files)

Citation/BibTeX parsing + conversion utilities leveraged by manuscripts and retrieval scripts.

6.1.19 `infrastructure.rendering` (120 files)

Pandoc shim, Unicode/XeLaTeX postprocessors, combined PDF/HTML/slide exporters.

6.1.20 `infrastructure.reporting` (411 files)

Parses pytest + coverage artefacts for dashboards; pairs with Stage 01 summaries and downstream executive exports.

6.1.21 `infrastructure.research` (3 files)

Seven-stage research workflow (SCOPE → LITERATURE → REASON → DESIGN → COMPUTE → SYNTHESIZE → WRITE) defined as typed ResearchStage data classes with explicit outputs, skills, and template commands per stage. Includes a full PRISMA-adapted literature review prompt and a research workflow prompt referencing template/ infrastructure commands.

6.1.22 `infrastructure.rules` (6 files)

Governance rules layer for validating project lifecycle transitions, sidecar sync policies, and pipeline gate orchestration. Rules mirror `projects/templates/` with git-tracked `templates/*` exemplars.

6.1.23 `infrastructure.scientific` (4 files)

Stability probing, benchmarking hooks—consumed heavily by optimization exemplars (`template_code_project` scripts).

6.1.24 `infrastructure.search` (77 files)

Two-tier search architecture: the `literature/` client stack (client.py, backends, caches) powers literature search with arXiv, Crossref, local, and Paperclip backends. `connectors/` exposes the built-in scientific database adapters through a uniform `ConnectorRegistry`; OpenAlex, UniProt, PDB, Semantic Scholar, European PMC, bioRxiv, and other registered adapters share list/search CLI commands with HTTP timeout,

retry, and TTL caching. `monid/` provides an opt-in paid gateway (`discover` / `inspect` / `run`) with offline USD/1k pricing reference in `monid/PRICING.md`. `deep_research/` dispatches long-horizon provider-neutral manuscript research (PAID, opt-in). The live registry, not this prose, is authoritative for connector count.

6.1.25 `infrastructure.sia` (10 files)

Generic Self-Improving-AI loop utilities: task-layout validation, execution harness, and metric capture reused by `template_sia` (fixture-replay by default).

6.1.26 `infrastructure.skills` (8 files)

Discovers `SKILL.md` frontmatter → `.agents/skill_manifest.json`.

6.1.27 `infrastructure.steganography` (13 files)

Watermark overlays, hashing companions triggered by secure pipeline path.

6.1.28 `infrastructure.tools` (6 files)

Invocable tool definitions registered by resource-pool governance; tools mirror `projects/templates/` with git-tracked `templates/*` exemplars.

6.1.29 `infrastructure.transmission` (6 files)

Rendered-release transmission leaves: LaTeX single-page bookends (`transmission_bookends`), page-check validation for Stage 04 (`transmission_page_check`), barcode strips, figure embedding, and shared models. Consumed by the publishing release workflow; regular package so the documentation module-reference guard resolves nested refs on disk.

6.1.30 `infrastructure.validation` (87 files)

Markdown + PDF + integrity CLIs underpinning Stage 04 diagnostics.

6.1.31 `infrastructure/logrotate.d` (0 files)

Operational templates for deployments (documentation-first; intentionally minimal Python footprint).

Documentation maturity: Coverage statements in Results pull from introspection—not hand-maintained denominators—so newly promoted modules automatically flow into manuscripts after `generate_manuscript_metrics.py`.
FAIR+RSE linkage: MCP-ready `SKILL.md` artefacts align with evaluator heuristics (executability + metadata richness) emphasized by FAIRsoft guidance [Garijo et al., 2024].

7 Security and Provenance

Research integrity requires more than reproducibility; it requires verifiable authorship. In an era of generative AI, automated scraping, and synthetic media, the ability to prove that a document was produced by a specific pipeline at a specific time is a critical defense against fabrication and misattribution. The W3C PROV data model [Moreau and Missier, 2013] establishes a formal vocabulary for expressing provenance records—entities, activities, and agents connected by derivation, generation, and attribution relations. Digital watermarking, pioneered by Cox et al. [Cox et al., 1997] for multimedia integrity verification, provides the foundational signal-processing theory for embedding imperceptible provenance markers within artifacts. `template/` implements a domain-specific provenance layer that embeds these relations directly into the PDF artifact itself, using four complementary steganographic and cryptographic mechanisms.

7.1 Threat Model

The steganography subsystem defends against three classes of threats:

1. **Unauthorized redistribution**: A manuscript is scraped and republished without attribution. The steganographic watermark survives the redistribution and can be used to prove original authorship.
2. **Content tampering**: Figures or text are modified after publication. The SHA-256 hash embedded in the PDF metadata detects any modification to the file contents.
3. **Provenance forgery**: An attacker claims to have produced a document they did not author. The build timestamp, commit hash, and pipeline metadata embedded in the watermark provide a verifiable chain of custody.

7.2 Steganographic Layers

The system applies four complementary layers of provenance information:

7.2.1 Layer 1: PDF Metadata Injection

The `inject_pdf_metadata` function writes structured metadata into both the PDF Info dictionary and an XMP (Extensible Metadata Platform) packet:

- `/Creator`: `Research Template Steganography Module`
- `/Producer`: Module path (`infrastructure.steganography`)
- `/CreationDate` / `/ModDate`: UTC timestamp in `ISO 8601` format
- `/Author`: From `config.yaml`
- `/Title`: From `config.yaml`
- `/Subject`: document identifier, plus `/DocumentID`
- Dedicated `/Hash_<ALGO>` fields for each computed digest (e.g. `/Hash_SHA256`)
- Custom fields: `GitCommit`, `GitCommitAvailable`, `/SteganographyTimestamp`
- An embedded `stego_manifest.json` attachment carrying the document id, digests, title, and git commit

7.2.2 Layer 2: Cryptographic Hashing

Before watermarking, SHA-256 (and SHA-512 when configured) digests of the original PDF are computed and stored in:

- A JSON hash manifest sidecar (`<pdf>.hashes.json`, e.g. `output/pdf/<name>_combined.hashes.json`)
- Dedicated PDF metadata fields (`/Hash_SHA256`, `/Hash_SHA512`)
- The embedded `stego_manifest.json` attachment

This enables post-hoc verification: anyone with the manifest can verify that the PDF has not been modified since rendering.

7.2.3 Layer 3: Alpha-Channel Overlay, Footer, and Invisible Text

A semi-transparent overlay is merged onto every page of the hardened PDF:

- **Watermark overlay**: in the default `text` mode, a configurable string (default `CONFIDENTIAL`) is repeated diagonally across the page; in `qr` mode a tiled QR overlay replaces it. The overlay is rendered at low opacity — the default is 8% (`overlay_opacity: 0.08`), configurable from 0.02 (subtle) to 0.30 (strong) — to be invisible during normal viewing but detectable through image analysis.
- **Provenance footer**: every page additionally receives a footer carrying the document identifier, page number / total pages, a digest prefix, title, authors, and source filename/size.
- **Invisible text layer**: the first page embeds a hidden text string with the machine-readable payload: `STEG_ID:<document-id>|TITLE:<title>|HASHES:<hash-prefix>`

The build timestamp and Git commit hash (short SHA) are recorded in the PDF metadata and the hash manifest rather than in the visible watermark text itself. Together these fields allow a verifier to reconstruct—from the hardened PDF alone—which version of the code, at which moment in time, produced the document.

7.2.4 Layer 4: QR and Barcode Injection

When barcodes are enabled, a bottom-of-page barcode strip (QR code + Code128 label) is merged onto **every** page, encoding the document identifier and page number (`ID:<doc-id-prefix>|P:<page>`). In `overlay_mode: "qr"`, a tiled QR overlay covers the full page in place of the text watermark. Both are configurable via the `steganography:` block in `manuscript/config.yaml` (or the repo-level `secure_config.yaml` defaults).

7.3 The `secure_run.sh` Orchestrator

The steganographic pipeline is orchestrated by `secure_run.sh`, a Bash script that wraps the standard `run.sh` pipeline with post-processing steganography:

1. Execute the standard YAML-declared pipeline (17 stages; default full 10) pipeline for the target project.
2. The `secure_run.sh` script invokes `SteganographyProcessor`.
3. Apply metadata injection, hashing, text overlay, and QR code injection.
4. Save the secured PDF alongside the original.
5. Generate a steganography report in JSON format.

The orchestrator processes either a single specified project or all discovered projects sequentially.

7.4 Tamper Detection

Verification is performed by comparing the stored SHA-256 hash against a freshly computed hash of the distributed PDF. Any modification—even a single bit flip—produces a hash mismatch. The alpha-channel overlay provides a secondary, visual verification channel that does not require access to the original hash.

7.5 Limitations

- Alpha-channel overlays are stripped by some PDF optimization tools (e.g., `qpdf --optimize`).
- QR codes are visible and may be removed by a determined attacker.
- The system does not provide non-repudiation in the cryptographic sense—it does not use digital signatures with private keys. Future versions may integrate GPG or X.509 signing.
- The provenance model is pipeline-centric rather than fully PROV-compliant. The path from `template/`'s current metadata-based provenance to full W3C PROV-compliant traces involves four steps: (1) *entity identification*—assigning stable identifiers (URIs or SWHIDs) to each pipeline input (manuscript files, data, config); (2) *activity logging*—recording each pipeline stage as a PROV Activity with start/end timestamps (already encoded in the watermark overlay); (3) *agent attribution*—binding each Activity to the pipeline version and Git commit hash (already encoded in the overlay and PDF metadata); (4) *PROV-O serialization*—emitting the provenance graph as OWL-RDF (PROV-O) or text (PROV-N) alongside the PDF. Steps (2) and (3) are already implemented; steps (1) and (4) are the primary remaining gaps. A future `infrastructure.provenance` module would close both gaps automatically.

7.6 Relationship to Software Supply Chain Integrity

The steganographic provenance layer operates at the *document level*—it certifies the integrity of a specific PDF artifact. A complementary concern is *build-level* provenance: certifying that the pipeline itself was executed with verified source code and dependencies. Frameworks such as in-toto [Torres-Arias et al., 2019] and SLSA (Supply-chain Levels for Software Artifacts) address this concern by defining attestation chains from source commit through build steps to final artifact. The NTIA's minimum elements for a Software Bill of Materials (SBOM) [National Telecommunications and Information Administration, 2021] further standardize the enumeration of software components and dependencies—essential for establishing the provenance lineage of build environments. SLSA defines four graduated levels of build integrity:

SLSA Level	Requirement	`template/` Status
1	Provenance document exists	✓ SHA-256 manifest + steganographic metadata
2	Version-controlled build scripts	✓ All scripts in git
3	Isolated build environment	about Docker support exists but not enforced in CI
4	Hermetic, reproducible builds	N – future work

Future versions of `template/` may generate SLSA-compatible provenance attestations alongside the steganographic watermarks, creating a two-layer provenance model: in-toto attests that the build pipeline was executed with the claimed source code, while the steganographic layer attests that the PDF was produced by that pipeline at a specific time.

7.7 Relationship to FAIR and Formal Provenance Standards

The steganographic layer supports the FAIR for Research Software (FAIR4RS) principles [Barker et al., 2022] at the artifact level. PDFs carry embedded metadata (Findability) in standardized XMP format (Interoperability). The SHA-256 hash manifest enables persistent integrity verification (a prerequisite for Reusability). The Documentation Duality standard ensures that the software producing the artifact is inspectable and well-documented (satisfying FAIRsoft [Garijo et al., 2024] metadata and documentation indicators). Full PROV-compliant provenance traces—capturing the derivation chain from source data through analysis scripts to rendered PDF—are a natural extension and a primary target for future development.

Software Heritage [Di Cosmo et al., 2020] complements this picture at the source-code level: by archiving the `template/` repository and assigning a reproducible SWHID (Software Hash Identifier) to each commit, Software Heritage makes the pipeline itself—not just its output—a citable, persistent digital artifact. A published SWHID alongside the PDF DOI creates a complete, two-artifact citation record: the paper's content is versioned via DOI; the code that generated it is versioned via SWHID. This combination satisfies the Katz et al. [Katz et al., 2021] software citation principles' requirement that software used in research be independently citable and permanently accessible.

8 Appendices

8.1 Appendix: Pipeline Stage Reference

8.1.1 Preparation and analysis

Table 1: Single-project DAG reference from default `pipeline.yaml` (names shown in dependency order). Scripts live under `scripts/`.

Stage name	Script / method	Primary inputs	Outputs / artefacts	Failure mode
Clean Output Directories	`_run_clean_outputs`	prior `projects/<name>/output/`, mirrored `output/<name>/` targets	emptied trees	Blocking
Environment Setup	`scripts/pipeline/stage_00_setup.py`	toolchain probes	scaffold dirs, env exports	Blocking
Infrastructure Tests	`scripts/pipeline/stage_01_test.py --infra-only --infra-scope pipeline-smoke`	`tests/infra_tests/`	smoke-test results	Required smoke tests must pass
Project Tests	`scripts/pipeline/stage_01_test.py --project-only`	`projects/<name>/tests/`	coverage artefacts	Nonempty tests, own coverage, verifier, and runner
Project Analysis	`scripts/pipeline/stage_02_analysis.py`	thin scripts	`figures/`, `data/`, reports	Blocking

8.1.2 Search, provenance and rendering

Stage name	Script / method	Primary inputs	Outputs / artefacts	Failure mode
Connector Search	`scripts/pipeline/stage_08_connector_search.py`	`manuscript/config.yaml`	`output/data/connector_search/`	opt-in (`science` tag); fresh policy-bound skip if unconfigured
Provenance Record	`scripts/pipeline/stage_09_provenance_record.py --stage "Connector Search"`	prior stage outputs	`.provenance/graph.json`	opt-in (`provenance` tag); fresh policy-bound skip if unconfigured
PDF Rendering	`scripts/pipeline/stage_03_render.py`	`manuscript/`, placeholders	`.pdf`/`.tex` bundles	Blocking

Stage name	Script / method	Primary inputs	Outputs / artefacts	Failure mode
Output Validation	`scripts/pipeline/stage_04_validate.py`	render tree	Markdown + PDF diagnostics JSON	Enabled formats, bookends, and provenance block; prose diagnostics advisory

8.1.3 Review and delivery

Stage name	Script / method	Primary inputs	Outputs / artefacts	Failure mode
LLM Manuscript Review	`scripts/pipeline/stage_06_llm_review.py --reviews-only`	resolved manuscript artefacts	textual reviews	Fresh policy-bound skip; required live acceptance remains blocking
LLM Translations	`scripts/pipeline/stage_06_llm_review.py --translations-only`	abstract metadata	multilingual snippets	Fresh policy-bound skip; required live acceptance remains blocking
Copy Outputs	`scripts/pipeline/stage_05_copy.py`	validated tree	mirrored `output/<name>/…`	Blocking

8.1.4 Optional packaging and publication

Stage name	Script / method	Primary inputs	Outputs / artefacts	Failure mode
Ebook Generation	`scripts/pipeline/stage_11_ebook.py`	rendered combined markdown	`output/ebook/` (EPUB/MOBI/DOCX)	opt-in (`ebook` tag); blocking when invoked
docxplus Export	`scripts/pipeline/stage_13_docxplus.py`	project source tree + metadata	`output/docxplus/` (.docx, .docxplus)	opt-in (`docxplus` tag); blocking when invoked
Metadata Package	`scripts/pipeline/stage_12_metadata.py`	`manuscript/config.yaml`	`output/metadata/` (ONIX/JSON/OPF)	opt-in (`metadata` tag); blocking when invoked

Stage name	Script / method	Primary inputs	Outputs / artefacts	Failure mode
Executable Bundle	`scripts/runner/bundle_executable.py`	project tree + outputs	container bundle manifest	opt-in (`bundle` tag)
Archival Publication	`scripts/runner/archive_publication.py`	bundle + deliverables	archival deposit manifest	opt-in (`archival` tag)

`scripts/pipeline/stage_07_executive_report.py` provides separate **multi-project executive reporting**, not an additional single-project DAG node. `execute_multi_project.py` writes aggregate summaries after project execution; this does not imply automatic invocation of that separate Stage 07 script.
Opt-in stages also prevent completion when invoked and unsuccessful. An allowed skip requires a fresh policy-bound reason receipt; it is not a successful execution. The executable DAG and its contracts remain authoritative.

8.2 Appendix: Configuration Reference

Table 2: Configuration schema for `config.yaml`, showing all supported fields and their structure.

```yaml
paper:
  title: "Paper Title"
  subtitle: "Optional Subtitle"
  version: "1.0"
  date: "2026-03-19"

authors:
  - name: "Author Name"
    orcid: "0000-0000-0000-0000"
    email: "author@example.com"
    affiliation: "Institution"
    corresponding: true

publication:
  doi: "10.5281/zenodo.XXXXXX"
  version_doi: "10.5281/zenodo.XXXXXX"
  version_record: "https://zenodo.org/records/XXXXXX"
  github_repository: "owner/repository"
  repository_url: "https://github.com/owner/repository"
  published_artifacts:
    github_pages: "https://owner.github.io/repository/"
    software_heritage: "https://archive.softwareheritage.org/..."
    pypi: "https://test.pypi.org/project/..."
    osf: "https://osf.io/..."
    huggingface_hub: "https://huggingface.co/datasets/owner/repository"
    netlify: "https://..."
    ipfs_pinata: "https://gateway.pinata.cloud/ipfs/..."
  transmission_bookends:
    enabled: false
    max_prior_releases: 5
    show_steganography: true
  journal: "Target Journal"
  volume: "1"
  pages: "1-10"
  year: "2026"

keywords:
  - "keyword1"
  - "keyword2"

metadata:
  license: "Apache License 2.0"
  language: "en"

llm:
  reviews:
    enabled: true
    types: [executive_summary, quality_review]
  translations:
    enabled: false
```

```yaml
testing:
  max_test_failures: 0
  max_infra_test_failures: 3
  max_project_test_failures: 0
```

8.3 Appendix: Repository Directory Structure

```
template/
|-- infrastructure/
|   |-- config/ docker/ documentation/ llm/
|   |-- orchestration/   # Thin Python entry equal to `./run.sh` backend
|   |-- prose/ reference/ rendering/ reporting/
|   |-- scientific/ search/ skills/ steganography/ validation/
|   |-- project/ core/
|   `-- logrotate.d/      # Operational rotation templates (no Python pkg)
|-- scripts/
|   |-- pipeline/          # stage_00_setup.py … stage_12_metadata.py (canonical, cited
by pipeline.yaml)
|   `-- runner/            # execute_pipeline.py execute_multi_project.py
bundle_executable.py archive_publication.py
|-- projects/                    # Typed program subfolders (`discover_projects`)
|   |-- templates/               # Public exemplars (git-tracked) - full roster:
|   |   |                        #   docs/_generated/active_projects.md (regenerated by
scripts/docgen/active_projects.py)
|   |   `-- template_template/   # Present manuscript (`manuscript/` here)
|   |-- active/                  # Hot-seat rendered set (symlinked, private)
|   |-- working/                 # Non-rendered backburner (symlinked, private)
|   |-- published/               # Non-rendered published (symlinked, private)
|   |-- archive/                 # Non-rendered retired (symlinked, private)
|   `-- other/                   # Non-rendered misc (symlinked, private)
|-- docs/ (20 top-level areas, 379+ markdown files per live counter)
|-- tests/                       # Infra suites (948+ files)
|-- AGENTS.md / README.md / CLAUDE.md / pyproject.toml
|-- run.sh / secure_run.sh
`-- output/ …                    # Mirrors after copy stage
```

See `docs/_generated/active_projects.md` for regenerated slugs (`uv run python scripts/docgen/active_projects.py`).

8.4 Appendix: Exemplar Project Summary

Table 3: Three representative workspaces under `projects/templates/` illustrating heterogeneous domains while sharing Layer 1. This is a hand-picked sample, not the full roster: the complete public exemplar set is enumerated dynamically from `PUBLIC_PROJECT_NAMES` and listed below.

The full public exemplar roster is: `templates/template_active_inference`, `templates/template_advanced_literature_review`, `templates/template_autopoiesis`, `templates/template_autoresearch_project`, `templates/template_autoscientists`, `templates/template_code_project`, `templates/template_data_descriptor`, `templates/template_eda_notebook`, `templates/template_experiment_tree`, `templates/template_formal`, `templates/template_gold_refinement`, `templates/template_literature_meta_analysis`, `templates/template_madlib`, `templates/template_methods_paper`, `templates/template_newspaper`, `templates/template_pitch_deck`, `templates/template_pools_rules_tools`, `templates/template_prose_project`, `templates/template_redacted_report`, `templates/template_registered_report`, `templates/template_search_project`, `templates/template_sia`, `templates/template_storybook`, `templates/template_template`, `templates/template_textbook`. The three rows below are a representative sample; a future `exemplar_summary_table` token in `build_manuscript_metrics_dict` would let this table cover every exemplar without hand-editing.

Project slug	Purpose	Highlights	Tests	Figures (Stage 02 hint)
`template_code_project`	Optimization tutorial	Convex demo figures, scripted tables	263 discovered tests; 90% declared gate	Controlled matplotlib exports
`template_prose_project`	Prose-heavy workflow	Validates narrative-only repos	165	Lightweight / optional plots
`template_autoresearch_project`	AutoResearch readiness	Planner + validation CLI	318	Readiness reports from Stage 02

Meta manuscript location: introspective study lives in `projects/templates/template_template/` beside the public exemplar set. Discovery now follows the typed `projects/` layout—`projects/templates/**` and `projects/active/**` are discovered/rendered, while `projects/working/**`, `projects/ongoing/**`, and `projects/archive/**` remain non-rendered—see root `CLAUDE.md` for invocation patterns (`resolve_project_root`).

8.5 Appendix: Documentation Inventory

The repository maintains documentation at three levels:

Table 4: Documentation inventory across the four-layer documentation architecture, from repository-wide system files to per-module skill descriptors.

Level	Files	Purpose
Repository root	`AGENTS.md`, `.agents/`, `CLAUDE.md`, `README.md`, `docs/` `RUN_GUIDE.md`	Global navigation and AI agent context
`docs/` directory	379 files across 20 subdirectories	User guides, API reference, troubleshooting
Per-directory	`AGENTS.md` + `README.md` in documented directories	Documentation Duality standard
Per-module (Tier 3)	`SKILL.md` on skill-enabled modules and public exemplars	Machine-parseable MCP-aligned skill descriptor
Infrastructure-level (PAI)	`PAI.md` at `infrastructure/` directory	Personal AI Infrastructure integration contract

The `docs/` subdirectories cover: `core/` (essential docs), `guides/` (progressive skill-level guides), `architecture/` (system design), `usage/` (content authoring), `operational/` (build, config, logging, troubleshooting), `reference/` (API, FAQ, glossary), `modules/` (31 infrastructure modules), `development/` (contributing, testing), `maintenance/` (long-horizon maintenance: private projects, CI-local, regression, archival, bundle), `best-practices/` (version control, migration), `plans/` (strategic plans and decision records), `rules/` (contributor norms), `prompts/` (22 AI prompt templates), `security/` (steganography, hashing), `streams/` (livestream notes), and `audit/` (review reports), plus the machine-generated `_generated/` snippets and `images/` assets.

Every count in this appendix is injected from live repository introspection rather than hand-maintained: `379` counts every Markdown file beneath `docs/` recursively, `20` counts its first-level subdirectories, and `22` counts the workflow subdirectories that each carry a `SKILL.md` descriptor. This is the same discipline the manuscript argues for throughout—a hand-typed documentation total silently rots as the tree grows, whereas a token re-resolves on every render. A reader onboarding to the repository should start at `docs/core/`, follow the graduated `docs/guides/` skill ladder, and consult the per-directory `AGENTS.md`/`README.md` pair nearest to whatever code they are editing; AI agents begin with root `AGENTS.md` and `.agents/AGENTS.md`, then read local contracts and available `SKILL.md` descriptors. The shared skill manifest supplies discovery; client-specific entrypoints reference the shared contracts.

8.6 Appendix: Comparative Tool Matrix

Symbol key (applies to all cells): **Y** = full native support · **about** = partial or plugin-based · **N** = absent. See the comparative feature matrix in the Results section for a colour-coded heatmap rendering of this table.

Table 5: Comparative feature matrix (14 capabilities × 10 tools). Y = full native support, about = partial or plugin-based, N = absent.

Capability	`templa te/`	Snake make	Nextfl ow	CW L	Quar to	Jupyter Book	R Mar kdown	DV C	Overleaf	OpenAI Prism
Pipeline orchestra- tion	Y	Y	Y	Y	about	N	N	Y	N	N
Manuscript rendering	Y	N	N	N	Y	Y	Y	N	Y	Y
Testing enforce- ment	Y	N	N	N	N	N	N	N	N	N
Coverage thresholds	Y	N	N	N	N	N	N	N	N	N
Cryptographic prove- nance	Y	N	about [1]	N	N	N	N	about [2]	N	N
Steganographic water- marking	Y	N	N	N	N	N	N	N	N	N
Multi- project manage- ment	Y	N	N	N	N	N	N	N	about	about
AI-agent documen- tation	Y	N	N	N	N	N	N	N	about	about
Agentic skill protocol (SKILL.md / MCP)	Y	N	N	N	N	N	N	N	N	N
Interactive TUI	Y	N	N	N	N	N	N	N	N	N
Zero-mock policy	Y	N	N	N	N	N	N	N	N	N
Container support	about	Y	Y	Y	N	N	N	N	N	N
Distributed execution	N	Y	Y	Y	N	N	N	about [3]	N	N
Multi- language (R/Julia)	N	Y	N	Y	Y	Y	Y	Y	N	N

[1] Nextflow lineage records operate at workflow scope rather than as a rendered-document citation graph. [2] DVC provides content-addressed versioning for data artifacts via its object store. [3] DVC integrates with

remote storage (S3, GCS, Azure) but does not natively orchestrate distributed compute. This matrix is a versioned manuscript snapshot, not a continuously updated product survey. A future refresh should re-check every external capability against current primary documentation before changing a cell.

References

J. J. Allaire, Charles Teague, Carlos Scheidegger, Yihui Xie, and Christophe Dervieux. Quarto, 2024. URL https://github.com/quarto-dev/quarto-cli.

Peter Amstutz, Michael R. Crusoe, Nebojša Tijanić, Brad Chapman, John Chilton, Michael Heber, Andrey Kartashov, Dan Leehr, Hervé Ménager, Maya Nedeljkovich, Matt Scales, Stian Soiland-Reyes, and Luka Stojanovic. Common workflow language, v1.0. *Specification, Common Workflow Language working group*, 2016. doi: 10.6084/m9.figshare.3115156.v2.

Anthropic. Introducing the model context protocol. Anthropic Engineering Blog, November 2024. URL https://www.anthropic.com/news/model-context-protocol. Open-source protocol for secure, two-way communication between AI models and external tools/data sources. Donated to the Linux Foundation, December 2025.

Monya Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533(7604):452–454, 2016. doi: 10.1038/533452a.

Lorena A. Barba. Terminologies for reproducible research. *arXiv preprint arXiv:1802.03311*, 2018. doi: 10.48550/arXiv.1802.03311.

Michelle Barker, Neil P. Chue Hong, Daniel S. Katz, Anna-Lena Lamprecht, Carlos Martinez-Ortiz, Fotis Psomopoulos, Jennifer Harrow, Leyla Jael Castro, Morane Gruenpeter, Paula Andrea Martinez, and Tom Honeyman. Introducing the FAIR principles for research software. *Scientific Data*, 9(1):622, 2022. doi: 10.1038/s41597-022-01710-x.

Carl Boettiger. An introduction to docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1):71–79, 2015. doi: 10.1145/2723872.2723882.

Jeremy Cohen, Daniel S. Katz, Michelle Barker, Neil P. Chue Hong, Robert Haines, and Caroline Jay. The four pillars of research software engineering. *IEEE Software*, 38(1):97–105, 2021. doi: 10.1109/MS.2020.2973362.

Ingemar J. Cox, Joe Kilian, F. Thomson Leighton, and Talal Shamoon. Secure spread spectrum watermarking for multimedia. *IEEE Transactions on Image Processing*, 6(12):1673–1687, 1997. doi: 10.1109/83.650120.

Roberto Di Cosmo, Morane Gruenpeter, and Stefano Zacchiroli. Referencing source code artifacts: A separate concern in software citation. *Computing in Science & Engineering*, 22(2):33–43, 2020. doi: 10.1109/MCSE.2019.2963148.

Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017. doi: 10.1038/nbt.3820.

Leonard P. Freedman, Iain M. Cockburn, and Timothy S. Simcoe. The economics of reproducibility in preclinical research. *PLOS Biology*, 13(6):e1002165, 2015. doi: 10.1371/journal.pbio.1002165.

Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995. ISBN 0-201-63361-2.

Daniel Garijo, Luis García, Matthias Barker, Rubén Chaves, Oscar Corcho, et al. FAIRsoft—a practical implementation of FAIR principles for research software. *Bioinformatics*, 40(8):btae464, 2024. doi: 10.1093/bioinformatics/btae464.

Robert Gentleman and Duncan Temple Lang. Statistical analyses and reproducible research. *Journal of Computational and Graphical Statistics*, 16(1):1–23, 2007. doi: 10.1198/106186007X178663.

Carole Goble, Sarah Cohen-Boulakia, Stian Soiland-Reyes, Daniel Garijo, Yolanda Gil, Michael R. Crusoe, Kristian Peters, and Daniel Schober. FAIR computational workflows. *Data Intelligence*, 2(1–2):108–121, 2020. doi: 10.1162/dint_a_00033.

Google. Agent2agent (A2A) protocol specification. Open Protocol Specification, 2025. URL https://a2a-protocol.org/latest/specification/. Open protocol for AI agent interoperability built on HTTP, JSON-RPC, and SSE; announced at Google Cloud Next '25, April 2025.

Morane Gruenpeter, Daniel S. Katz, Anna-Lena Lamprecht, Tom Honeyman, Daniel Garijo, Alexander Struck, Anna Niehues, Paula Andrea Martinez, Leyla Jael Castro, Tovo Rabemanantsoa, Neil P. Chue Hong, Carlos Martinez-Ortiz, Laurents Sesink, Matthias Liffers, et al. Defining research software: A controversial discussion. In *SSRN Preprint*, 2021. doi: 10.2139/ssrn.3884311.

Tom Honeyman, Michelle Barker, Daniel S. Katz, and Neil P. Chue Hong. The FAIR principles for research software: Three years on. Research Data Alliance Virtual Plenary 24, 2024. URL https://www.rd-alliance.org/wp-content/uploads/2025/04/RDA_VP24_FAIR4RS_Three_Years_On_Reformatted.pdf. Two-year retrospective review recommending principle amendments including explicit reproducibility requirement and clarification of domain-relevant standards.

Iterative, Inc. DVC: Data version control. https://dvc.org, 2024. Open-source tool for ML experiment management, data versioning, and pipeline orchestration.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://arxiv.org/abs/2310.06770.

Daniel S. Katz, Neil P. Chue Hong, Tim Clark, Mercè Crosas, Bohdan B. Khomtchouk, John E. Kratz, David Leinweber, Kyle Niemeyer, Arfon Smith, and Patrick Vandewalle. Recognizing the value of software: A software citation guide. *F1000Research*, 9:1257, 2021. doi: 10.12688/f1000research.26932.2.

Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. Jupyter notebooks — a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90, 2016. doi: 10.3233/978-1-61499-649-1-87.

Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984. doi: 10.1093/comjnl/27.2.97.

Johannes Köster and Sven Rahmann. Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19):2520–2522, 2012. doi: 10.1093/bioinformatics/bts480.

Leslie Lamport. *LaTeX: A Document Preparation System*. Addison-Wesley, 2nd edition, 1994. ISBN 978-0201529838.

Anna-Lena Lamprecht, Leyla Garcia, Mateusz Kuzak, Carlos Martinez, Ricardo Arcila, Eva Martin Del Pico, Victoria Dominguez Del Angel, Stephanie van de Sandt, Jon Ison, Paula Andrea Martinez, Peter McQuilton, Alfonso Valencia, Jennifer Harrow, Fotis Psomopoulos, Josep Ll. Gelpi, Neil Chue Hong, Carole Goble, and Salvador Capella-Gutierrez. Towards FAIR principles for research software. *Data Science*, 3(1):37–59, 2020. doi: 10.3233/DS-190026.

Sam Lau and Philip J. Guo. The design space of LLM-based AI coding assistants: An analysis of 90 systems. In *IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2025. URL https://pg.ucsd.edu/publications/ai-coding-assistants-design-space_VLHCC-2025.pdf.

Chris Lu, Jeff Clune, Jakob Foerster, Brian Lim, Robert Tjarko Lange, Yutian Tang, Shengran Mao, et al. The AI scientist: Towards fully automated open-ended scientific discovery. arXiv preprint arXiv:2408.06292, 2024. URL https://arxiv.org/abs/2408.06292.

Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008. ISBN 978-0132350884.

Gerard Meszaros. *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley, 2007. ISBN 978-0131495050.

Luc Moreau and Paolo Missier. PROV-DM: The PROV data model. W3c recommendation, World Wide Web Consortium, 2013. URL https://www.w3.org/TR/prov-dm/.

Laurenz Mädje and Martin Haug. Typst: A new markup-based typesetting system, 2023. URL https://typst.app. Programmable typesetting system with incremental compilation; open-sourced 2023.

National Telecommunications and Information Administration. The minimum elements for a software bill of materials (SBOM). Technical report, U.S. Department of Commerce, 2021. URL https://www.ntia.gov/sites/default/files/publications/sbom_minimum_elements_report_0.pdf. Defines minimum SBOM fields: supplier, component name, version, unique identifier, dependency relationship, author, timestamp.

Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114.

Daniel Nüst, Markus Konkol, Edzer Pebesma, Christian Kray, Marc Schutzeichel, Holger Przibytzin, and Jörg Lorenz. Opening the publication process with executable research compendia. *D-Lib Magazine*, 23 (1/2), 2017. doi: 10.1045/january2017-nuest.

Open Source Security Foundation. SLSA: Supply-chain levels for software artifacts, v1.0. https://slsa.dev/spec/v1.0/, 2023. Accessed: 2026-03-19.

OpenAI. Prism: AI-assisted research writing environment. https://openai.com/prism, 2026. Launched 2026-01-27 on GPT-5.2; provides context-aware manuscript assistance across notes, drafts, equations, and citations.

Overleaf (Digital Science). Overleaf: Collaborative cloud LaTeX editor. https://www.overleaf.com, 2025. Real-time collaborative LaTeX editor used by 15M+ researchers across 4,000+ institutions.

Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847.

Stephen R. Piccolo and Michael B. Frampton. Tools and techniques for computational reproducibility. *GigaScience*, 5(1):30, 2016. doi: 10.1186/s13742-016-0135-4.

João Felipe Pimentel, Leonardo Murta, Vanessa Braganholo, and Juliana Freire. A large-scale study about quality and reproducibility of Jupyter notebooks. In *Proceedings of the 35th IEEE/ACM International Conference on Software Maintenance and Evolution (ICSME)*, pages 507–517, 2019. doi: 10.1109/ICSME.2019.00080.

Karthik Ram. Git can facilitate greater reproducibility and increased transparency in science. *Source Code for Biology and Medicine*, 8(1):7, 2013. doi: 10.1186/1751-0473-8-7.

Research Software Alliance. Developing actionable guidelines for FAIR research software. ReSA Task Force, 2024. URL https://zenodo.org/records/18877929. Task Force launched December 2024 analyzing 17 FAIR4RS principles into 6 actionable categories; builds on FAIR-BioRS methodology.

Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.1003285.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:2302.04761, 2023. URL https://arxiv.org/abs/2302.04761.

Eric Schulte, Dan Davison, Thomas Dye, and Carsten Dominik. A multi-language computing environment for literate programming and reproducible research. *Journal of Statistical Software*, 46(3):1–24, 2012. doi: 10.18637/jss.v046.i03.

Joseph P. Simmons, Leif D. Nelson, and Uri Simonsohn. False-positive psychology: Undisclosed flexibility in data collection and analysis allows presenting anything as significant. *Psychological Science*, 22(11):1359–1366, 2011. doi: 10.1177/0956797611417632.

Victoria Stodden, Marcia McNutt, David H. Bailey, Ewa Deelman, Yolanda Gil, Brooks Hanson, Michael A. Heroux, John P. A. Ioannidis, and Michela Taufer. Enhancing reproducibility for computational methods. *Science*, 354(6317):1240–1241, 2016. doi: 10.1126/science.aah6168.

Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, pages 1393–1410, 2019. URL https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023. URL https://arxiv.org/abs/2305.16291.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024a. doi: 10.1007/s11704-024-40231-1.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yunzhu Song, Bowen Li, Jasmine Li, Tao Zhang, Chuyi Ma, Ruohan Yu, Wangchunshu Yin, Karen Livescu, and Graham Neubig. OpenDevin: An open platform for AI software developers as generalist agents. arXiv preprint arXiv:2407.16741; published as OpenHands at ICLR 2025, 2024b. URL https://arxiv.org/abs/2407.16741.

Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, Jan-Willem Boiten, Luiz Bonino da Silva Santos, Philip E. Bourne, et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi: 10.1038/sdata.2016.18.

Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6):e1005510, 2017. doi: 10.1371/journal.pcbi.1005510.

Yihui Xie, J. J. Allaire, and Garrett Grolemund. *R Markdown: The Definitive Guide*. Chapman and Hall/CRC, 2018. ISBN 978-1138359338. doi: 10.1201/9781138359444.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023. URL https://arxiv.org/abs/2210.03629.

END OF TRANSMISSION

Release: v2.0.0 · DOI 10.5281/zenodo.20419007 · SHA-256 pending… · pairing pending

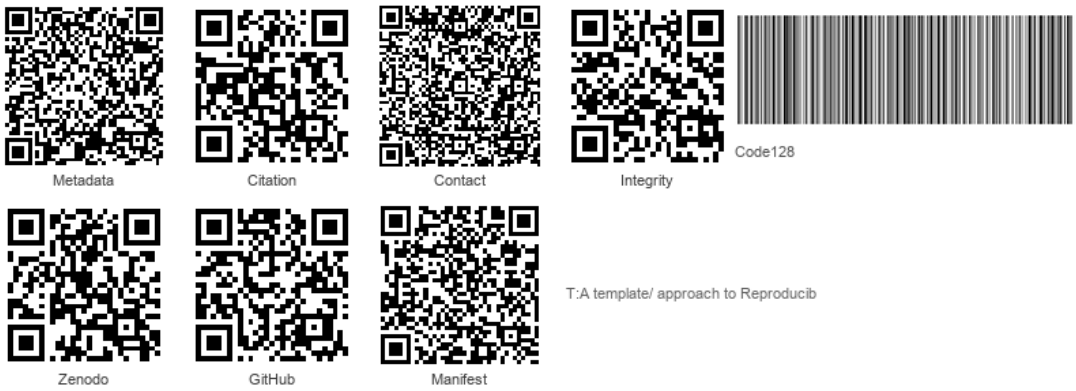


Figure 7: Integrity QR strip

Prior: v1.0.8 · 10.5281/zenodo.20932076 · b9bc5cf3…